信州大学 SHINSHU UNIVERSITY
SHINSHU UNIVERSITY
GRADUATE SCHOOL OF SCIENCE AND TECHNOLOGY

令和 7 年度 修士論文

量子化学計算における固有値問題の FPGA アクセラレーションに関する研究

指導教員: 佐藤 真平 准教授

令和 8 年 1 月

提出者

信州大学 大学院総合理工学研究科 工学専攻
情報数理・融合システム分野

24W6056K　長澤駿

概要

本論文は，量子化学計算における主要な計算負荷である行列の固有値問題に対し，修正グラム–シュミット (MGS) 法に基づく QR 分解を用いた FPGA による高速化手法を提案するものである．

量子化学計算とは，シュレディンガー方程式を計算機で近似的に解くことで，分子の電子状態・構造・物性・反応性などを原子レベルで予測する手法である．代表的な密度汎関数法 (DFT) では反復計算の各ステップで行列の固有値問題を繰り返し解く必要があり，大規模化に伴う行列サイズの増大により固有値計算がボトルネックとなる．量子化学計算の高速化をすることで．より大規模で高精度なシミュレーションが可能になる．

本研究では，固有値計算アルゴリズムである QR 法において支配的となる QR 分解を FPGA で高速化して，固有値計算全体の高速化を図る．先行研究 [1] の単精度浮動小数点 MGS-QR 分解アーキテクチャを Agilex 7 FPGA 上に再現し，これを比較基準（ベースライン）とする．先行研究の分析から，行列次数 N の増加に伴い内積計算回数が $O(N^2)$ で増大し，そのレイテンシが全体性能を支配することが分かる．このボトルネックを解消するため，内積演算器を二重化したアーキテクチャを提案する．これにより，内側ループでのデータ供給と内積演算を並列化し, 1 サイクル当たりの内積処理スループットを向上させて, 計算全体の低レイテンシ化を図る．

このように，FPGA に負荷の大きい計算をオフロードすることで，低レイテンシ化やスループットの向上が期待できる．

評価の結果，提案手法はベースラインと比較して，行列サイズ $N \geq 128$ で実行サイクル数および計算コア実行時間を削減し，カーネル実行時間も $N \geq 64$ で短縮する．計算コア実行時間の削減に伴い, GFLOPS の性能向上を達成する．

結論として，提案手法により，MGS-QR 分解の計算コア実行時間とカーネル実行時間を削減し，GFLOPS の性能の向上を達成する．

目次

概要

第 1 章　序論 **1**

1.1　研究背景 1

1.2　研究目的 2

第 2 章　前提知識 **3**

2.1　量子化学計算の概要 3

2.1.1　量子化学計算とは 3

2.1.2　HF 法と DFT 法 4

2.1.3　SCF 計算 4

2.2　固有値問題 5

2.2.1　固有値アルゴリズムの概観 5

2.2.2　QR 法 8

2.2.3　Gram–Schmidt　QR 分解 8

2.3　FPGA 10

2.3.1　FPGA 10

2.3.2　Intel 製 FPGA の主要リソース 12

2.3.3　High Level Synthesis 16

2.3.4　Intel oneAPI による開発 18

第 3 章　関連研究 **20**

3.1　先行研究の提案手法 20

3.1.1　アルゴリズム 20

3.1.2　アーキテクチャ 22

3.1.3 先行研究の性能評価方法 22
3.1.4 先行研究の性能・リソース 24

第 4 章 ベースラインの実装 26
4.1 再現方針と対象 26
4.1.1 再現の目的 26
4.1.2 再現条件（数値形式・行列サイズ） 26
4.1.3 再現範囲と差異 27
4.1.4 再現度の評価指標 27
4.2 実装 27
4.2.1 設計・開発環境 27
4.2.2 アーキテクチャの対応付け 27
4.3 妥当性の確認 29
4.3.1 リソース使用量・周波数の比較 29
4.4 ベースライン性能 31
4.4.1 ベースライン性能の比較結果 31
4.4.2 小括 32

第 5 章 提案手法 33
5.1 先行研究のボトルネック分析 33
5.1.1 内積計算レイテンシの支配性 33
5.2 本研究で解決する先行研究の課題 34
5.3 アルゴリズム選択の理由 34
5.3.1 MGS-QRD の計算・アクセス特性 34
5.4 提案設計 35
5.4.1 内積計算器の二重化 35
5.4.2 オンチップメモリ構成 35
5.5 提案アーキテクチャ 36
5.5.1 FPGA カーネル内部の動作 37
5.5.2 FPGA グローバルメモリとのデータ転送 38
5.5.3 アルゴリズムに基づく詳細動作 38

第 6 章 評価 40
6.1 合成結果の比較と考察 40

6.1.1 DSP 使用量 40
6.1.2 ALM 41
6.1.3 周波数への影響 41
6.2 演算器のレイテンシ 41
6.3 計算コア実行時間とカーネル実行時間の比較 42
6.3.1 計算コア実行時間 42
6.3.2 カーネル実行時間の比較 43
6.3.3 GFLOPS の導出 43
6.4 CPU・GPU との比較 44

第 7 章 結論 **47**
7.1 結論 47
7.1.1 今後の課題 48

謝辞 **49**

参考文献 **51**

図目次

2.1 対称行列向け固有値解法 6
2.2 FPGA の設計フロー 13
2.3 ALM ブロック図 14
2.4 浮動小数点 32 ビット単精度モードでの概略ブロック図 15
2.5 高位合成での設計フロー 17

3.1 先行研究のアーキテクチャ図 23

4.1 ベースラインのカーネル構成 28

5.1 行列の分割方式とバンキング 36
5.2 提案アーキテクチャ 37

6.1 GFLOPS の比較（文献値は各図からの概算） 45
6.2 実行時間の比較（FPGA は Kernel 実測，文献値は図から概算．A100 は GFLOPS から $2N^3$ で換算した参考値） 46

表目次

2.1 固有値計算アルゴリズム 7
2.2 直交変換の手法 7

3.1 Sustained-to-Peak and absolute performance より抜粋 25
3.2 先行研究のリソース表（Table 2 より抜粋） 25
3.3 先行研究における各演算器レイテンシの比較（Table 1 より抜粋） 25

4.1 設計・開発環境 28
4.2 Baseline 実装と先行研究のリソース比較 30
4.3 Baseline 実装と先行研究における各演算器レイテンシの比較 31
4.4 vector Depth および Real Latency の推定値（Baseline と先行研究の比較） 32

6.1 合成結果の比較（ベースライン vs 提案手法） 41
6.2 Baseline 実装と提案手法における各演算器レイテンシの比較（cycle） . . 42
6.3 Baseline 実装と提案手法の周波数・Real Latency・計算コア実行時間の比較（差分付き） 42
6.4 カーネル実行時間の比較（ベースライン vs 提案手法） 43
6.5 Baseline 実装と提案手法の GFLOPS 比較 44

第 1 章

序論

1.1 研究背景

量子化学計算とは，シュレディンガー方程式を近似的に解くことで化学現象をシミュレーションする方法である．シュレディンガー方程式の解は波動関数である．この波動関数を求めることで，分子の電子状態・構造・反応性などを予測・理解することができる [2]．これらを明らかにすることで，材料科学，物理学，生物科学，創薬といった分野で量子化学計算は役立っている [3][4].

しかし，シュレディンガー方程式の厳密解を求めることはほとんどの場合において不可能である．そこで，ハートリー・フォック (HF) 法や密度汎関数法 (DFT) といった手法を用いることで，シュレディンガー方程式を近似的に解く [5]．量子化学計算を高速化することで，大規模で高精度なシミュレーションが可能になる．大規模化では実際の材料やデバイスに近いスケールでの挙動予測が可能になる [6]．また，タンパク質や溶媒中の薬剤分子などの複雑な分子のシミュレーションも可能になる [7][8]．高精度化では半導体のバンドギャップ予測などに役立つ [9].

量子化学計算は対象の分子の原子数に応じて計算コストが増加することが知られている．HF 法や DFT 法では，原子数 N に対して $\boldsymbol{O(N^3)}$〜$\boldsymbol{O(N^4)}$ の計算コストを持つとされている [3]．また，通常の DFT 法の計算では数百原子を超えると計算コストが重くなり始めることが報告されている [10]．このコストの増加は，原子数の増加に伴って計算で扱う行列のサイズが大きくなることに起因する．

HF 法や DFT 法では精度を高くすると．行列のサイズが大きくなる．精度を高くするには分子の軌道を正確に表現することが量子化学計算では重要である．分子の軌道を正確に表すにはより多くの軌道を用いる必要がある．この軌道の組が，行列のサイズの増加に影

響する．

また，HF 法や DFT 法では固有値問題（対角化）を繰り返し解く必要がある．HF 法や DFT 法では特定の収束条件を満たすまで，固有値問題を繰り返し解くことで，最終的な解を得る手法になっている．

したがって，HF 法や DFT 法の計算過程における行列の固有値問題がボトルネックの一つとなる．これは，精度よく，かつ，原子数が大きい分子を計算対象にすると，サイズの大きい行列の固有値問題を解く必要があるためである．この原子数の増加による対角化のコスト増加は [11][12] においても課題とされている．

そこで，固有値問題の高速化という研究分野を概観し，Field Programmable Gate Array(FPGA) を用いて固有値問題をアクセラレーションするアプローチをとる．FPGA は論理回路を用途に合わせて再構成できる特徴を持つ．用途に合わせて演算器やデータパスを特化して構成することで，高い並列性を活用した処理を実現できるデバイスである．この特徴を活かし，固有値問題のアルゴリズムに合わせて FPGA を構成する．これにより，固有値問題の高スループット化または低レイテンシ化が期待できる．

FPGA に固有値問題をオフロードすることで，実行時間の削減につながり，量子化学計算の高速化が期待できる．オフロードを達成することで，CPU は固有値問題以外の処理に集中することができる．

1.2 研究目的

本研究では，先行研究 [1] を元に提案手法を論ずる．先行研究 [1] は，修正グラムシュミット (MGS)QR 分解を FPGA 向けのアルゴリズムに改良して実装した．この実装ではアルゴリズムの内積計算と乗算-減算をオーバーラップさせることで，実行性能/理論性能 (sustained-to-peak) が 95% を達成するアーキテクチャを提案している．

先行研究の手法では，内積計算の処理回数によるレイテンシが全体の性能に影響を与えることが問題である．そこで本研究では，内積計算器の二重化とそれに対応したメモリブロック構成をアーキテクチャとして提案する．このアーキテクチャにより，計算コアの実行時間や FPGA カーネルの時間の削減を実現する．

提案アーキテクチャはハイエンド FPGA である Agilex7 を搭載した IPAC-1000 ボードに実装する．比較対象として，先行研究のアーキテクチャを同 FPGA ボードに実装することで実機評価を行う．実機による性能を CPU や GPU で実装された QR 分解と比較することで，提案手法の有効性を確認する．

第 2 章

前提知識

本章では，量子化学計算を概説したうえで，提案手法に関係する固有値問題のアルゴリズムと FPGA の構造や設計環境について説明する．

2.1 量子化学計算の概要

2.1.1 量子化学計算とは

量子化学計算はコンピュータを用いて自然現象をシミュレーションする「計算科学」の分野に属する．量子化学計算は分子の電子状態を計算することで，実験を行わずに分子の性質を予測・理解するために用いられる．具体的には，分子の最適化構造，スペクトルなどの分子物性の予測，分子の電子状態，化学反応の解析をすることができる．また，計算科学には扱う対象のサイズ (原子レベルからマクロな物体まで) や時間スケールに応じて，量子化学計算以外にも様々な手法があり，量子化学計算が「電子」の振る舞いに着目するのに対し，原子核の動きや，より大規模な原子の集団を扱う手法が存在する．量子化学計算以外の代表的なシミュレーション手法としては，分子動力学法や固体物理計算がある．

量子化学計算はシュレディンガー方程式の解である波動関数を近似的に解く試みである．シュレディンガー方程式は

$$\boldsymbol{H\Psi} = E\boldsymbol{\Psi} \tag{2.1}$$

式 (2.1) で表される．ここで，$\boldsymbol{H}$(ハミルトニアン) は分子や原子のエネルギーを表す演算子であり，原子の種類と位置，および電子数の情報が含まれている．$\boldsymbol{\Psi}$ は波動関数を表し，電子の分布や性質を決めるもとになる．E は波動関数に対応するエネルギー (スカラー) である．シュレディンガー方程式ではある全電子の波動関数を求めることになるが，ほとん

どの場合において厳密な解を得ることはできない．したがって，シュレディンガー方程式を近似的に解く，ハートリー・フォック (HF) 法や密度汎関数法 (DFT 法) が主に量子化学計算では用いられる．

2.1.2　HF 法と DFT 法

HF 法は全電子の波動関数を求めるシュレディンガー方程式とは違って，１電子波動関数についての方程式である．HF 方程式を解く．なお，次式 2.2 が HF 方程式である．f はハミルトニアンを近似したフォック演算子を表し，ε_k は軌道エネルギー，ψ_k は１電子の分子軌道である．HF 法では，シュレディンガー方程式の電子間の相互作用を個別で考えずに，ある電子とそのほかの電子がつくる電場との相互作用として近似することで，N 電子波動関数問題を１電子波動関数の方程式に帰着する．

$$f\psi_k = \varepsilon_k\psi_k \tag{2.2}$$

DFT 法では波動関数の代わりに電子密度に着目した，コーンシャム (Kohn-Sham) 方程式を解く．コーン・シャム方程式は電子密度においてシュレディンガー方程式と等価なため，HF 法では近似解を求めるのに対し，密度汎関数法は原理的には真の解を得ることができる．密度汎関数法は HF 法と同程度の計算コストであるが，より高精度な解を得ることができるので広く利用されている．[2] 実際には密度汎関数法では式中に現れる「交換・相関汎関数」の正確な形が不明なため，近似解となる．なお，コーン・シャム方程式は HF 方程式と類似しており，変数の定義が変わった 2.3 で表される．

$$f'\psi'_k = \varepsilon'_k\psi'_k \tag{2.3}$$

f' は近似されたハミルトニアンである Kohn–Sham フォック演算子，ε'_k は Kohn–Sham 軌道エネルギー，ψ'_k は Kohn–Sham 軌道を表す．

2.1.3　SCF 計算

式 (2.2) と式 (2.3) は方程式が非線形 (演算子が自身の解に依存する形) であるため，SCF(Self-Consistent-Field：自己無撞着場計算) と呼ばれる反復プロセスを実行する．
以下がプロセスである．

1. 初期の分子軌道を計算
2. HF 方程式またはコーン・シャム方程式を解く

3. エネルギー，分子軌道を計算する
4. 収束判定．収束しなければ，２に戻る

なお，２の段階で HF 方程式とコーン・シャム方程式は

$$\boldsymbol{FC} = \boldsymbol{SC}\varepsilon \tag{2.4}$$

式 (2.4) で表される Roothaan 方程式という一般化固有値問題に帰着された形で計算される．この Roothaan 方程式は，HF 方程式とコーン・シャム方程式の演算子を行列として表現した式になっている．収束判定では現在と一つ前のイテレーションで得たエネルギーの差を求め，差が閾値以下であれば収束とする．したがって，HF 法と DFT 法では SCF 計算によって，固有値計算が反復的に実行されるアルゴリズムとなっている．

2.2 固有値問題

2.2.1 固有値アルゴリズムの概観

固有値計算では基本的に反復法を用いる必要がある．固有値計算の基本原理は「対象の行列に相似変換を施しても固有値が不変」である．故に固有値を求めることは相似変換を何度も作用させて対角行列にすること (対角化) である．しかしこの操作は行列の値に依存し，定められた回数の演算で終了する保証はない．そこで，固有値計算アルゴリズムでは，解が許容誤差内に収束するまで繰り返す反復法という手法が用いられている．

アルゴリズムは大きく以下の３つの方法がある．

(1) 相似変換を繰り返して，行列 A を対角化する
(2) 定数回の相似変換で行列 A を扱いやすい行列に変換する前処理を施してから固有値を求める
(3) 行列 A をそのまま扱って固有値・固有ベクトルを求める

密行列向けのアルゴリズムでは (1) のような反復的な手法と，(2) のような定数回の相似変換で三重対角行列に変換し，三重対角化に対する固有値を計算する反復的な手法がある．(1) のようなアルゴリズムにはヤコビ法と QR 法がある．ヤコビ法は非対角要素を 0 にすることで，対角行列に変換する古典的な手法である．QR 法は行列を直交行列 Q と上三角行列 R に分解し，それを逆順に乗ずる (RQ) 操作を繰り返すことで対角行列に変換する手法である．(2) の方法での前処理ではハウスホルダー (Householder) 変換とギブンス回転を使った三重対角化手法があり，ハウスホルダー変換が数値の安定性から一般的である [13].

三重対角行列に対する固有値・固有ベクトルの計算アルゴリズムは QR 法，２分法，分割統治法 (Divide and Conquer method)，MRRR(Multiple Relatively Robust Representation) 法などがある．ただし，三重対角化を経由する場合は「三重対角化した行列の固有ベクトル」から「密行列の固有ベクトル」を復元する手順が必要で，ハウスホルダー逆変換で真の固有ベクトルを得る．この計算では $O(n^3)$ の演算が必要である．また，元の行列，変換の累積，固有ベクトルなどを格納するために約３倍のメモリ領域が必要である．したがって，三重対角化を経由する固有値計算にはハウスホルダー変換を使った三重対角化と逆変換に時間を費やす．以下にアルゴリズムをまとめた表 2.1，2.2 と図 2.1 を記す．

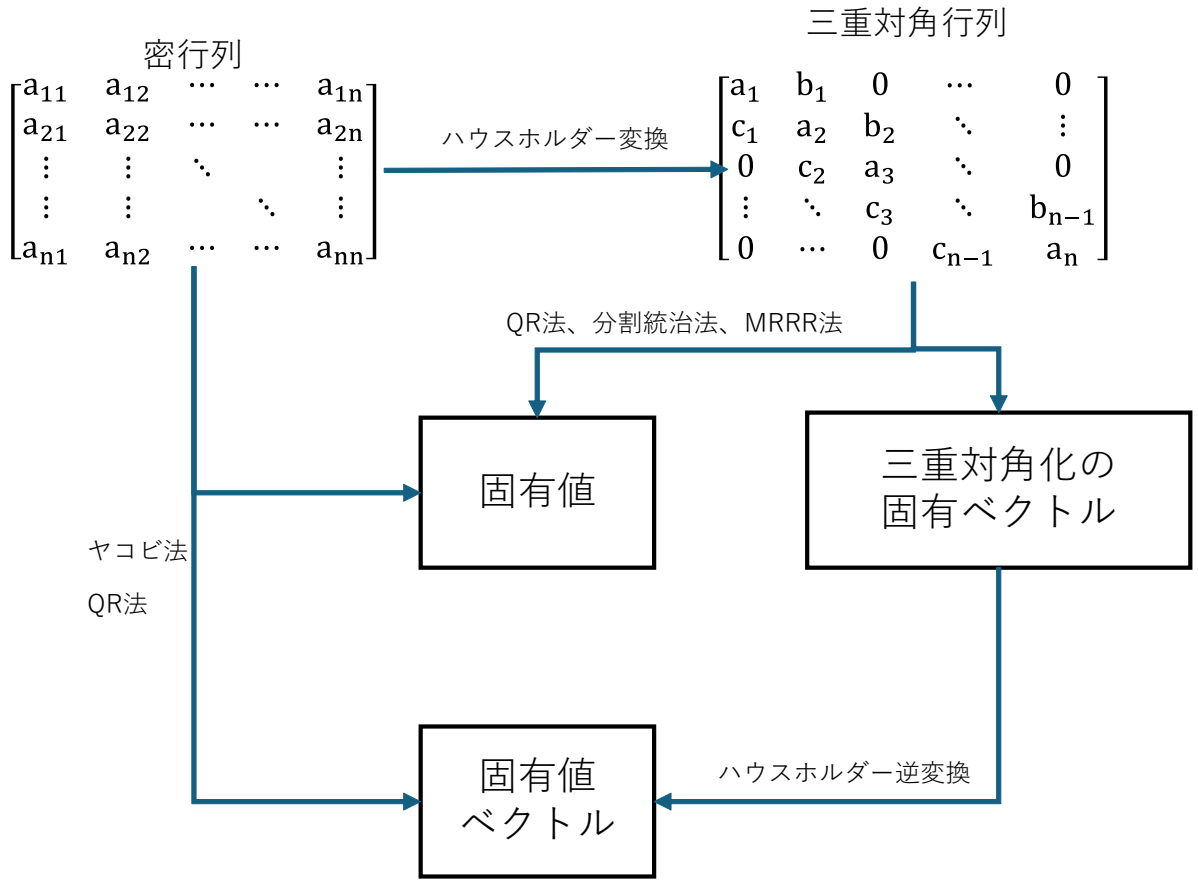


図 2.1　対称行列向け固有値解法

表 2.1 固有値計算アルゴリズム

アルゴリズム	対象	処理内容
ヤコビ法	対称	ヤコビ回転で非対角要素を 0 にする．全固有値・固有ベクトルを計算.
QR 法	実対称・実三重対角	QR 分解・実三重対角化で対角化．全固有値を計算.
分割統治法	実三重対角	行列を部分問題に分割し，その解を統合することで解を得る．全固有値・固有ベクトルを高速に計算.
MRRR 法	三重対角	全固有値・固有ベクトルを高速・高精度に計算.

表 2.2 直交変換の手法

アルゴリズム	対象	処理内容
ハウスホルダー	対称 (密)	対称行列をハウスホルダー行列で三重対角行列に変換
ギブンス変換	実対称	実対称行列をギブンス回転行列で三重対角行列に変換

2.2.2 QR 法

QR 法は密行列の QR 分解で得られる直交行列 Q による直交変換を繰り返すことで，行列を対称行列または上三角行列に収束させ，全固有値を計算する手法である．以下が QR 法の流れである．

1. A を入力に取り，A^0 とする
2. 行列 A を QR 分解する．次式の k は繰り返しの回数である．

$$A^k = Q^k R^k \tag{2.5}$$

3. 2で求めた Q^k と R^k を逆に乗じて A を更新する．

$$A^{k+1} = R^k Q^k \tag{2.6}$$

上記１，２を繰り返すことで A^{k+1} の非対角要素が０に近づいていき，上三角行列に収束することで全固有値が対角線上に現れる．

2.2.3 Gram–Schmidt　QR 分解

QR 法で実行される QR 分解は，グラムシュミット直交化法をすることで達成できる．グラムシュミットの直交化法は n × n 行列 $\boldsymbol{A}$ の n 次元列ベクトル $a_1, a_2, ..., a_n$ が線形独立の時，正規直交基底 $\{q_1, q_2, ...q_n\}$ を抽出するアルゴリズムである．つまり，$\boldsymbol{A}$ の列ベクトルを順番に直交化して $\boldsymbol{Q}$ を作り，係数として $\boldsymbol{R}$ を得る方法である．以下の式 2.7，2.8 に従うことで直交基底 q_n は計算することができる．

$$\boldsymbol{a'_n} = \boldsymbol{a_n} - \sum_{i=1}^{n-1} \langle \boldsymbol{a_n}, \boldsymbol{q_i} \rangle \boldsymbol{q_i} \tag{2.7}$$

$$\boldsymbol{q_n} = \frac{\boldsymbol{a'_n}}{\|\boldsymbol{a'_n}\|} \tag{2.8}$$

式 (2.5)，(2.6) について各 $\boldsymbol{a_n}$ について解くと

$$\begin{aligned}
\boldsymbol{a_1} &= \|\boldsymbol{a'_1}\| \boldsymbol{q_1} \\
\boldsymbol{a_2} &= \langle \boldsymbol{a_2}, \boldsymbol{q_1} \rangle \boldsymbol{q_1} + \|\boldsymbol{a'_2}\| \boldsymbol{q_2} \\
&\vdots \\
\boldsymbol{a_n} &= \langle \boldsymbol{a_n}, \boldsymbol{q_1} \rangle \boldsymbol{q_1} + \langle \boldsymbol{a_n}, \boldsymbol{q_2} \rangle \boldsymbol{q_2} + \cdots + \langle \boldsymbol{a_n}, \boldsymbol{q_{n-1}} \rangle \boldsymbol{q_{n-1}} + \|\boldsymbol{a'_n}\| \boldsymbol{q_n}
\end{aligned}$$

となり，行列の形式で表現すると

$$A = QR = (\boldsymbol{q}_1\ \boldsymbol{q}_2\ \cdots\ \boldsymbol{q}_n)\begin{pmatrix} \|\boldsymbol{a}_1'\| & \langle \boldsymbol{a}_2, \boldsymbol{q}_1\rangle & \cdots & \langle \boldsymbol{a}_n, \boldsymbol{q}_1\rangle \\ 0 & \|\boldsymbol{a}_2'\| & \ddots & \vdots \\ \vdots & \ddots & \ddots & \langle \boldsymbol{a}_n, \boldsymbol{q}_{n-1}\rangle \\ 0 & \cdots & 0 & \|\boldsymbol{a}_n'\| \end{pmatrix} \tag{2.9}$$

式 (2.9) の形で書くことができる．定義通り，Q は正規直交な列ベクトル $\boldsymbol{q_n}$ を並べたもので直交行列，R は対角成分より下が 0 な上三角行列になる．

グラムシュミットの直交法には古典的なグラムシュミット法 (CGS:Classical Gram–Schmidt) と修正グラムシュミット法 (MGS:Modified Gram–Schmidt) が存在する．MGS の方が丸め誤差の影響を受けにくいことが知られている．以下に CGS の疑似コードと，[1] に記載の MGS の疑似コードを記す．主なコードの違いは CGS では列ベクトルの内積計算 (Algorithm 1: L8) と更新 (Algorithm 1 L11) を二段階に分けるのに対し，MGS は列ごとに内積と更新を連続して行う (Algorithm 2: L8, L9)．この構造により MGS では直交化を逐次的に行い，更新後の残差に基づいて次の係数を計算するため，CGS の一括直交化に比べて有限精度演算下で直交性が保たれやすい．

Algorithm 1 Classical Gram–Schmidt based QR factorization

1: **function** QRD(A)
2: **Input :** Matrix A (square), $A = [a_1, \ldots, a_n]$
3: **Output:** Matrix Q (orthogonal) and R (upper-triangular)
4: **for** $i = 1$ **to** n **do**
5: $r_{i,i} = \|a_i\|_2$
6: $q_i = a_i / r_{i,i}$
7: **for** $j = i + 1$ **to** n **do**
8: $r_{i,j} = \langle q_i, a_j \rangle$
9: **for** $j = i + 1$ **to** n **do**
10: $a_j = a_j - r_{i,j}\, q_i$

Algorithm 2 Modified Gram–Schmidt–based QR factorization (from[1])

function QRD(A)
  Input : Matrix A (square), $A = [a_1, \ldots, a_n]$
  Output: Matrix Q (orthogonal) and R (upper-triangular)
  for $i = 1$ **to** n **do**
    $r_{i,i} = \|a_i\|_2$
    $q_i = a_i / r_{i,i}$
    for $j = i + 1$ **to** n **do**
      $r_{i,j} = \langle q_i, a_j \rangle$
      $a_j = a_j - r_{i,j}\, q_i$

2.3 FPGA

2.3.1 FPGA

Field Programmable Gate Array（FPGA）は，論理仕様をプログラムすることで，ユーザー所望の論理回路を実現できる再構成可能な半導体デバイスである [14]．FPGA は多数のプログラマブルな論理要素から構成され，その中心となる Look Up Table（LUT）は入力パターンに対する出力（真理値表）を定義することで任意の論理関数を実現する．ユーザが設計した回路は，この LUT の内容および周辺のフリップフロップ（FF）や配線設定としてコンフィグレーションデータ（ビットストリーム）に変換され，デバイス内部の設定メモリへ書き込まれることで，回路構造そのもの（組合せ回路・逐次回路，パイプライン，並列化構造など）をハードウェアとして実装できる．これにより，CPU や GPU のように固定された命令セットを逐次実行するプロセッサとは異なり，用途に応じてデータパス幅や演算器配置，ストリーミング処理，専用パイプラインなどをカスタム化し，低レイテンシ処理や高スループット処理を狙った設計が可能となる．

また，FPGA は特定用途向け集積回路（ASIC）などのカスタム IC と比較して，製造工程におけるマスクセットの作成やそれに伴う初期費用が不要であり，設計変更を手元で即座に反映できる点が利点である．設計の反復や仕様変更への追従が容易であるため，研究開発や試作段階における実機評価，あるいは運用中の機能更新が求められる用途において有効な選択肢となる．なお，多くの FPGA は SRAM ベースであり電源断で設定が消失するため，起動時に外部フラッシュやホストからビットストリームをロードして動作を開始

する．

FPGA は基本構成要素として，プログラマブルな論理ブロック，外部インターフェースを提供するプログラマブルな入出力（I/O）要素，それらを接続するプログラマブルな配線要素から構成される．多くの FPGA はアイランドスタイルと呼ばれる構造を持ち，論理ブロックを中心に，論理ブロックと配線チャネルを接続するコネクションブロック，配線チャネル同士の接続先を切り替えるスイッチブロックを組み合わせたタイルが格子状に配置されている．論理ブロック内部は一般に LUT とフリップフロップ（FF）を中心とするロジックセル（ベンダによって LE/CLB/ALM など名称が異なる）からなり，LUT により任意の論理関数を実現し，FF により状態保持やパイプラインレジスタを構成する．I/O 要素は外部信号との電気的仕様（電圧規格，終端，スルーレート等）や入出力タイミングを満たすための機能を担い，ボード上のメモリや周辺回路，ホストと接続する．配線要素は配線セグメントとスイッチ行列の集合として実装され，論理ブロック・I/O・ハードマクロ間の接続関係をユーザ設計に応じて構成する．これらの要素が二次元的に配置され，配置配線（Place & Route）によって「どの論理ブロックを使い」「どの配線で結ぶか」が決定されることで，所望の回路が形成される．

FPGA では演算性能を上げるためのハードウェアブロックとして DSP やエンベデッドメモリを備える．DSP（Digital Signal Processing）ブロックはデジタル信号処理に適した専用演算器であり，内部に加算器・乗算器・レジスタを持つことで，乗算，加算，および積和演算（MAC）を高速に実行できる．乗算器を LUT のみで構成すると，多数の論理ブロックを消費し，かつ配線遅延の増大により動作周波数が制約されやすいが，DSP ブロックを用いればこれらの演算を専用ハードウェアに集約できるため，性能向上と資源効率の両立が容易になる．さらに，乗算・加算など計算集約部分を DSP へ割り当てることで，汎用の論理ブロックを制御やデータ整形など他の処理に充てられる．一方，エンベデッドメモリは FPGA 内部に搭載されたメモリブロックであり，ベンダによって BRAM（Block RAM）や M20K などの名称で提供される．FPGA アクセラレータでは外部メモリ（DDR/HBM 等）との転送帯域がボトルネックとなり性能を制約することが多いため，低レイテンシでアクセス可能なエンベデッドメモリを作業領域として用いることが重要である．エンベデッドメモリはバンキングにより複数ブロックへ並列アクセスでき，大きなオンチップ帯域を確保できるほか，設計に応じて FIFO として構成することも可能である．なお，DSP やエンベデッドメモリは，ベンダが推奨する演算パターンや配列アクセス形態，属性指定等の記述スタイルに従ってコーディングすることで，設計ツールにこれらハードマクロの使用を推論させ，適切な資源へマッピングできる．

FPGA のコンフィグレーションデータ（ビットストリーム）は，ハードウェア記述言語

によるレジスタ転送レベル（RTL）の記述を起点として，図 2.2 複数段階のフローを経て生成される．まず，ユーザが Verilog/VHDL/SystemVerilog 等で記述した RTL は，論理合成（Synthesis）により論理ゲートやレジスタからなる中間表現（ネットリスト）へ変換される．次にテクノロジーマッピング（Technology Mapping）により，ゲートレベルのネットリストがデバイス固有の基本素子（LUT/FF，DSP，BRAM 等）へ割り当てられる．続いて配置配線（Place & Route）では，割り当てられた素子をチップ上の物理位置へ配置し，プログラマブル配線資源を用いて接続を成立させる．この段階で配線遅延が確定し，静的タイミング解析（STA）により制約周波数（Fmax）やセットアップ／ホールド条件が満たされているかが評価される．最終的に，配置配線結果に基づいて生成されるビットストリームが起動時にデバイスへロードされ，論理回路として動作を開始する．なお，実設計ではタイミング制約（クロック定義，入出力制約，マルチサイクル等）や検証（RTL シミュレーション，オンボード検証）を反復しながら，周波数・資源・性能の目標を満たすよう最適化を進める．さらに近年では，C/C++ 言語などの高級言語で動作を記述し，RTL を自動生成する高位合成（HLS: High-Level Synthesis）も広く用いられる．高位合成では，C/C++ で記述した動作仕様が Verilog/VHDL 等の HDL（あるいは中間表現としての RTL）へ変換され，その後は RTL 設計と同様に論理合成，テクノロジーマッピング，配置配線のフローを経てビットストリームが生成される．

2.3.2 Intel 製 FPGA の主要リソース

Adaptive Logic Module (ALM)

Adaptive Logic Module（ALM）は，Intel 製 FPGA を構成する最も基本的なプログラム可能ロジックブロックであり，ユーザが記述した回路（HDL や HLS で記述された演算・制御）を実デバイス上に実装する際の主要な割当て先となる．合成ツールはソースコードから生成された論理・算術・レジスタ回路を ALM へマッピングし，配線資源によって ALM 同士を接続することで，ユーザ所望のカスタム回路を構築する．

図 2.3，ALM は Adaptive LUT（AdaptiveLUT）と呼ばれる 8 入力の LUT 系資源を中心に，算術演算に有利な 2 つの加算器（キャリー系の専用回路を含む），および 4 つのレジスタから構成される．AdaptiveLUT は分割（fracturable）して複数の小規模論理関数を同一 ALM 内に実装できるため，制御ロジックの集積や局所配線の短縮に寄与しやすい．これにより，ALM は論理演算だけでなく，比較的小規模な加算・減算などの算術演算も効率的に実装できる．

数値演算を実装する際，FPGA はデジタル信号処理に特化した DSP ブロック（乗算加算

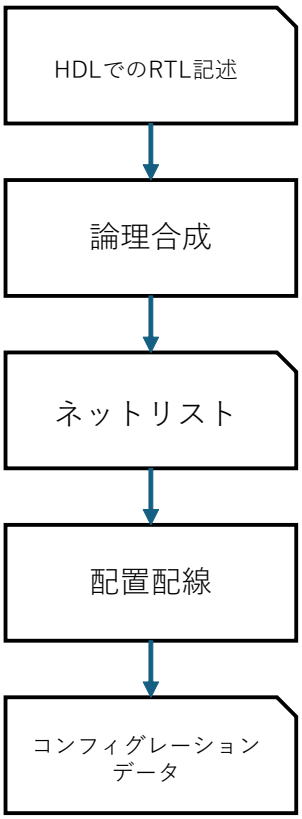


図 2.2 FPGA の設計フロー

器などの専用ハードウェア）を利用できる一方で，ALM のみを用いて算術回路を構成することも可能である．後者のように，DSP などの専用演算ブロックを使わずに ALM ベースで演算回路を構成する方式は一般にソフトロジック（soft logic）と呼ばれる．実装方針は，性能（Fmax，スループット，レイテンシ）や資源制約（DSP 残量，ALM 残量）に依存し，ソースコード側の指示子（例：演算の記述形態，HLS/コンパイラ指示）やツール設定により，DSP 優先またはソフトロジック優先の割当て方針を選択できる場合がある．

さらに ALM は，論理・算術回路としてだけでなく，小規模なメモリ（RAM）として活用できる点も重要である．ALM をメモリとして構成したものは MLAB（Memory LAB）と呼ばれ，比較的小さなデータ集合を低レイテンシに保持する用途（キュー，FIFO，テーブル，局所バッファ等）で有効である．したがって，ALM は演算回路の規模と性能だけでなく，局所メモリ構成にも関与するため，FPGA 設計におけるパフォーマンスおよびリソース効率を左右する主要要素となる．

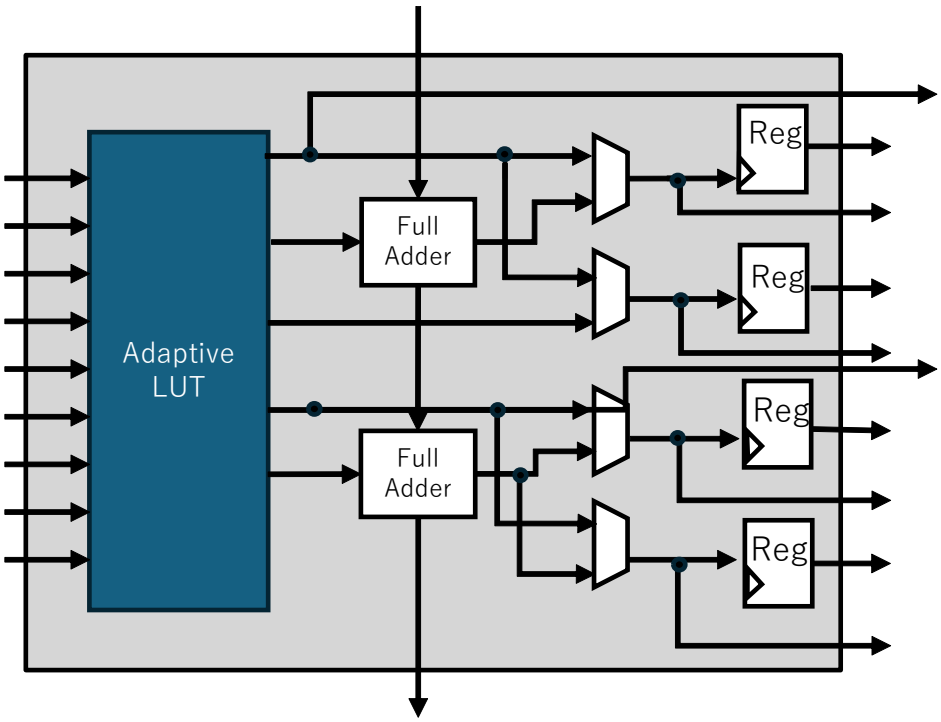


図 2.3 ALM ブロック図（元：Fig. 6）[15]

Intel 可変精度 DSP

Intel（Altera）FPGA に搭載される可変精度 DSP（Variable Precision DSP）ブロックは，乗算器・加算器・累算器などをハードウェアとして集積した演算ブロックであり，積和演算（MAC）を高スループットかつ高効率に実行するために最適化されている．可変精度 DSP は，固定小数点演算と浮動小数点演算の双方をサポートし，用途や必要精度に応じて演算精度・資源利用・性能のトレードオフを設計時に選択できる．以下では，浮動小数点モードの主な機能を述べた後に，浮動小数点モードにおける DSP 内部アーキテクチャを概説する．

可変精度 DSP の浮動小数点演算モードは，単精度（FP32）演算に加えて，半精度（FP16）および BFLOAT16 といった低精度形式を対象とする演算モードを備え，用途に応じて性能と精度のトレードオフを選択できる．FP32 モードでは，1 つの DSP ブロックで FP32 の乗算，加算／減算，積和（乗算＋加算），積差（乗算－減算），および乗累算（multiply-

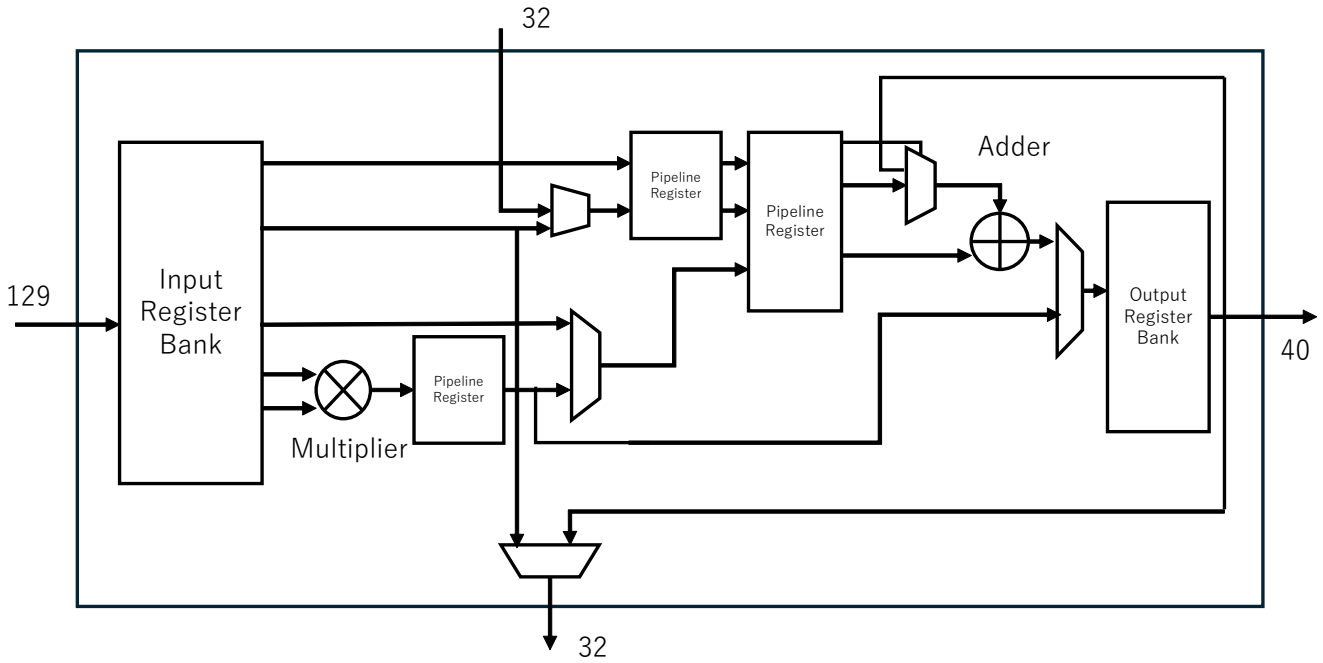


図 2.4　浮動小数点 32 ビット単精度モードでの概略ブロック図（元：Figure 5）[16]

accumulate）といった基本演算を実行できる．加えて，DSP ブロックが備えるチェーンイン（chain-in）／チェーンアウト（chain-out）を用いることで，別の DSP ブロックで生成された演算結果を直接取り込み，加算・減算として後段の DSP で合成できる．さらに低精度側では，1 つの DSP ブロック内で 2 つの FP16 乗算を並列に実行し，それらの和を 32 ビット結果（FP32）として生成する演算が可能である．また，この 32 ビット結果に対して単精度（FP32）のオペランドを加算する混合精度モードも提供されており，低精度乗算を活用しつつ加算を単精度で行う実装方針を取りやすい．[16]

図 2.4 に示すように，可変精度 DSP の浮動小数点アーキテクチャ（FP32 モード）は，入力レジスタバンク，IEEE754 準拠の単精度乗算器・加算器，複数段のパイプラインレジスタ，および出力レジスタバンクを一体化したデータパスとして構成されている．演算途中にパイプラインレジスタを配置することで高い動作周波数とスループットを両立しつつ，DSP ブロック間のチェーン（カスケード）接続を可能にし，前段 DSP の演算結果を後段 DSP で直接取り込んで加算・減算（および累算）に利用できる．

2.3.3 High Level Synthesis

高位合成（High Level Synthesis：HLS）は，C 言語や C++ などの高位プログラミング言語で記述されたアルゴリズム（動作記述）から，ハードウェア記述言語（HDL）による RTL（レジスタ転送レベル）記述を自動的に生成する技術である．従来の RTL 設計に比べて高い抽象度で設計を進められるため，設計期間の短縮やアルゴリズム検証の効率化に寄与し，近年は実用化が進む設計手法として広く利用されている．さらに HLS では，同一のソースコードに対してプラグマやディレクティブといったツールへの指示を調整することで，回路規模（面積）と動作速度（性能）のトレードオフを比較的容易に探索できる．これにより，詳細なタイミング設計やステートマシン設計など，RTL で手作業になりがちな工程の負担を軽減しつつ，所望の性能・資源制約を満たす実装へ段階的に収束させやすくなる．

また，近年の HLS 開発環境としては，Intel の oneAPI や AMD の Vitis などが代表例として挙げられる．これらのツールでは，ソフトウェア処理のうち「FPGA に計算させたい」部分を関数として切り出し，ツール側で当該関数を合成対象として指定することで，カーネル周辺のインタフェース回路やホスト側との接続，さらにホスト側の制御コードや実行環境との連携に必要な枠組みを整備できる（実際にはボードサポートやランタイムが提供する要素と組み合わせて利用する）．この結果，計算負荷の高い処理のみを FPGA へオフロードし，CPU は別の処理を並行実行する，といったヘテロジニアスな役割分担を比較的少ない実装コストで実現できるようになる．

HLS を用いた設計フローは，ソフトウェアレベルでの機能検証と，RTL 生成後のハードウェア観点での検証・最適化を段階的に進める点に特徴がある．設計フローの全体像を図 2.5 に示す．一般的な流れは次のとおりである．

1. C/C++ でのソースコード作成：回路化後のデータパスやメモリアクセスを意識しつつ，アルゴリズムを高位言語で記述する．
2. ソフトウェアエミュレーション：作成したソースコードがアルゴリズムとして正しいかをソフトウェア的に検証する．Intel oneAPI を用いた設計ではこの段階を FPGA エミュレーション，Vitis では C simulation（Csim）と呼ぶ.
3. 高位合成（HLS）：高位合成を実行して RTL/HDL を生成する．この時点で，生成物に基づくリソース見積もりや，タスク（ループ）スケジューリングなどの解析結果を確認できる．

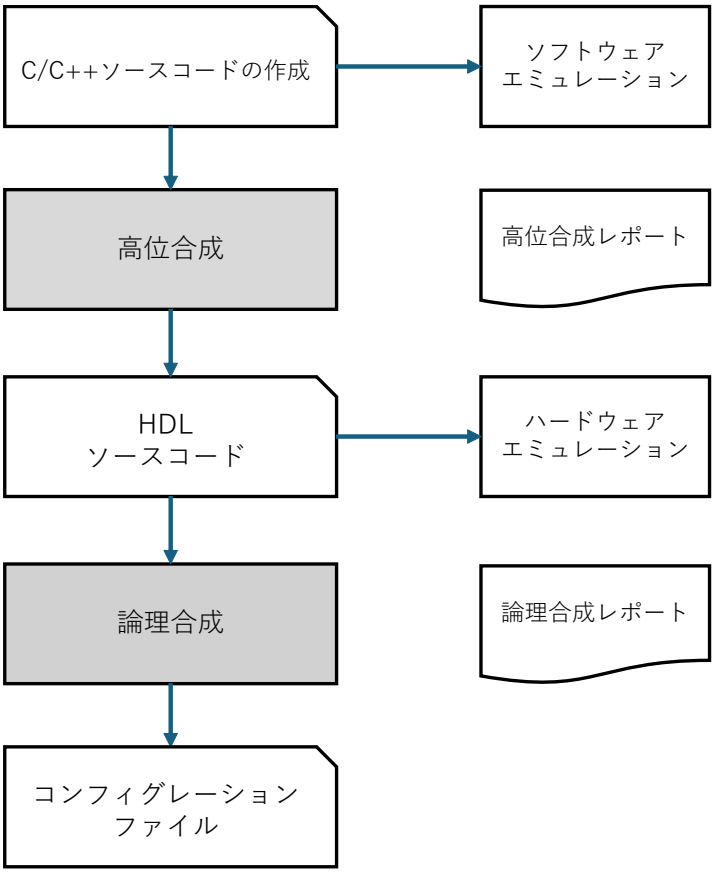


図 2.5 高位合成での設計フロー

4. ハードウェアエミュレーション：生成した RTL/HDL を用いて，ハードウェア挙動を意識した検証を行う．高位合成による RTL 変換の結果が反映されるため，パイプライン化やメモリアクセス，依存関係といった回路起因の振る舞いを確認しやすい．
5. 論理合成：RTL/HDL を対象として論理合成（必要に応じて配置配線まで）を行い，デバイス上で実現可能な回路へ落とし込む．この段階でタイミングや資源使用量がより具体化し，最終的な実装に向けた調整の指針が得られる．

一方で，高位合成を使った設計では，FPGA の構造を意識したハードウェア指向のアルゴリズムが重要である．高位合成は C 言語などの高位言語で記述できるが，CPU 向けに書かれたソフトウェアをそのまま高位合成ツールでコンパイルすれば，所望の性能で自動的に FPGA にマッピングされるわけではない．FPGA は命令逐次実行を前提とする CPU とは異なり，演算器やメモリを空間的に配置して並列に動作させるデバイスである．そのため，性能は利用可能な演算資源数，メモリ帯域，オンチップメモリ容量，配線遅延，なら

びにデータ依存関係や資源競合の有無といった制約の下で決まる．例えば，計算自体は並列化できてもメモリアクセスが帯域律速であればスループットが頭打ちになり，またループに依存関係が残るとパイプライン開始間隔（II）が増加して期待した性能が得られない．

したがって，高位合成では FPGA の並列性（パイプライン化，ループ展開，データフロー化）と制約（メモリアクセスのボトルネック，依存関係，資源競合）を意識した形でソースコードを記述し，ハードウェア指向のアルゴリズムへ落とし込む必要がある．その上で，プラグマやディレクティブにより，II の制御や並列度の指定，配列分割・バンキングなどのメモリ構成の誘導を行うことで，回路規模（面積）と性能のトレードオフを体系的に探索できる．すなわち高位合成は，「高位言語で書ける」こと自体が目的ではなく，ハードウェア指向のアルゴリズム設計とツール指示を組み合わせることで，FPGA 上での最適な実装へ収束させるための設計手段である．

2.3.4 Intel oneAPI による開発

Intel oneAPI は，CPU・GPU・FPGA などアーキテクチャの異なるハードウェアから構成されるヘテロジニアス・システムを，単一のプログラミング・モデルおよびコードベースで開発できるようにするための統合開発環境である [17][18]．Intel oneAPI では Data Parallel C++（DPC++）を用いることで，ホスト（CPU）側で実行される処理と，アクセラレータ側で実行される処理を単一の C++ ソースファイル（single-source）として記述できる．DPC++ の基盤となる SYCL は，C++ を拡張して異種混合プロセッサ向けの並列プログラミングを可能にするクロスプラットフォームな抽象化レイヤであり，デバイス選択やデータ管理，カーネル起動といったヘテロジニアス実行に必要な機能を言語・API として提供する．DPC++ は SYCL 規格の実装であると同時に，インテル独自の拡張機能を含むため，Intel oneAPI DPC++/C++ Compiler がこれを実装するコンパイラに相当する．さらに，Intel oneAPI では DPC++ によりアルゴリズムを記述することで，FPGA 向けにはコンフィグレーションデータ（ビットストリーム）や実行バイナリの生成までを同一の開発フローで扱える．

Intel oneAPI のメリットは，単一の言語（C++）で異種ハードウェア向けアプリケーションをプログラムできる点と，ライブラリ活用による開発効率および性能向上にある．ホストコードとデバイスコードを同一の C++ ソースファイル内に記述できるため，コード分割による管理負担が減り，型安全性を保ったまま共通化・再利用がしやすい．また，コンパイラによる最適化を同一の言語体系の中で適用しやすく，性能移植性（performance portability）の観点でも利点がある．加えて Intel oneAPI には，各ハードウェア向けに最

適化されたライブラリ群が用意されており，数値演算ライブラリである oneMKL，ディープラーニング向けの oneDNN，マルチコア CPU 向けのタスク並列処理ライブラリである oneTBB などを利用することで，アルゴリズムの実装負担を抑えつつ高性能化を図れる．

第 3 章

関連研究

3.1 先行研究の提案手法

先行研究 [1] は単精度浮動小数点の MGS(Modified Gram Schmidt)QR 分解を FPGA 向けに再構成した並列実行構造を提案し，FPGA に実装した．256 × 256〜512 × 512 の行列において，演算器・パイプラインのレイテンシを含む実行サイクル数（Real Latency）が理想下限（Peak Latency）に近く，実効性能/理論性能（sustained-to-peak）で 95% 以上を達成した．本研究では，この MGS-QRD のアーキテクチャを基盤として提案手法を設計した．そこで本節では，先行研究 [1] における MGS アルゴリズムおよびアーキテクチャを説明する．

3.1.1 アルゴリズム

先行研究 [1] は Algorithm 2 のデータ依存性と各演算のレイテンシについて言及している．まず，Algorithm 2 では外側ループでは列ベクトルのノルム計算が必要で，ノルム計算には内積計算と平方根計算が含まれている．これら演算は長いレイテンシと，次の除算にデータ依存を発生させ，約 45 サイクルの待機が発生する．また，内側ループの内積計算も乗算-減算操作に対して長いレイテンシとデータ依存を引き起こす．これらのデータ依存性により，Algorithm 2 では sustained-to-peak が 3% 程度にとどまると評価している．

そこで，先行研究 [1] は [19] で提案したアルゴリズムを基盤として Algorithm 3 を提案した．なお，このアルゴリズムでは式 3.1 を用いる．この式では，Algorithm 2 の列ベクトル $\boldsymbol{a_j}$ の計算において，$r_{i,j}$ と q_i の定義を代入することで計算の依存関係を減らすことがで

Algorithm 3 Proposed QRD algorithm

1: **function** QRD(A)
Input : Matrix A (square)
Output: Matrix Q (orthogonal) and R (upper-triangular)
2: $p_{1,1} = \langle a_1, a_1 \rangle$
3: $ir_{1,1} = \dfrac{1}{\sqrt{p_{1,1}}}$ ▷ via the reciprocal square-root
4: **for** $j = 2 : n$ **do**
5: $p_{1,j} = \langle a_1, a_j \rangle$
6: $s_{1,j} = \dfrac{p_{1,j}}{p_{1,1}}$ ▷ via the divider
7: $r_{1,j} = p_{1,j} \times ir_{1,1}$ ▷ via the multiplier
8: **for** $i = 1 : n - 1$ **do**
9: $q_i = a_i \times ir_{i,i}$
10: **for** $j = i + 1 : n$ **do**
11: $a_j = a_j - s_{i,j} a_i$
12: **if** $j = i + 1$ **then**
13: $p_{i+1,i+1} = \langle a_{i+1}, a_{i+1} \rangle$
14: $ir_{i+1,i+1} = \dfrac{1}{\sqrt{p_{i+1,i+1}}}$
15: $r_{i+1,i+1} = \sqrt{p_{i+1,i+1}}$
16: **else**
17: $p_{i+1,j} = \langle a_{i+1}, a_j \rangle$
18: $s_{i+1,j} = \dfrac{p_{i+1,j}}{p_{i+1,i+1}}$
19: $r_{i+1,j} = p_{i+1,j} \times ir_{i+1,i+1}$
20: $q_n = a_n \times ir_{n,n}$

きる．

$$a_j = a_j - \frac{\langle a_i, a_j \rangle}{\|a_i\| \, \|a_i\|} = a_j - \frac{\langle a_i, a_j \rangle}{\langle a_i, a_i \rangle} a_i \tag{3.1}$$

式 (3.1) によって，Algorithm 3 の構造では乗算-減算計算の結果を内積計算に供給するループ構造になっている．このループ構造によって乗算ー減算器と内積計算器が同時に動作することで効率を向上させている．Algorithm 3 は [19] で提案された，乗算-減算計算の結果を内積計算に供給するアルゴリズムをマッピングに関する考慮事項を含めて更新したものである．

Algorithm 3 を実装したアーキテクチャが次の図 3.1 を説明する．まず，このアーキテクチャでは Algorithm 3 での列ベクトルの内積計算と乗算加算を完全ループ展開して実行する．よって，列ベクトル $\boldsymbol{a_n} = [a_0, a_1, ...a_{n-1}]^T$ の n 要素同士の乗算，加減算は並列計算される．また，この並列計算のために，FPGA の RAM ブロックから列ベクトル $\boldsymbol{a_n} = [a_0, a_1, ...a_{n-1}]^T$ の n 個要素を並列に read する必要がある．これを実現するために $n \times n$ の行列要素を行優先でバンキングする．これにより RAM ブロックには 1 read/write で n 要素アクセスできる．この時，図 3.1 の内積計算器と乗算-減算器は DSP にマッピングする．内積計算の結果 $p_{i,j}$ は Algorithm 3 １０～１９行目の内側ループの最初のイテレーションでレジスタに格納し，以降のイテレーションで除算器や乗算器の入力となる．図 3.1 の黄色の演算器は主に大部分はソフトロジックで構成し，DSP へのマッピングは最小限に抑える．

3.1.2 アーキテクチャ

次にデータパスを説明する．このアーキテクチャではパイプラインのプレループ (Algorithm 3:L2 7) とメインループ (Algorithm 3:L8 20) で段階を分けることができる．プレループ時では RAM ブロックから列ベクトルを順次 read，乗算-減算器へ流す．乗算-減算器 $(x - \alpha y)$ では α を 0 にすることで，列ベクトルをバイパスさせて，内積計算器の入力とする．$\boldsymbol{a_1}$ は乗算-減算器と内積計算器の前のレジスタに格納する．内積計算の結果は逆平方根，除算，平方根，乗算器への入力となる．これらの $ir_{i,i}$ と $s_{i,j}$ の結果は FIFO に格納され，メインループで消費される．メインループではプレループと同様に列ベクトルを順次読み出す．乗算-減算器には FIFO から $ir_{i,i}$ や $s_{i,j}$ を読み出す．$\boldsymbol{q_i}$,$\boldsymbol{a_j}$ は計算され次第，RAM ブロックに書き出す．更新した $\boldsymbol{a_j}$ はそのまま内積計算に供給する．内積計算以降のデータパスはプレループと同様である．このアーキテクチャはパイプラインで動作する．故に，メインループでは列ベクトルの read，乗算-減算計算，内積計算，その他計算 (逆平方根，除算，平方根，乗算) がオーバーラップして実行される．

3.1.3 先行研究の性能評価方法

先行研究 [1] は上記アーキテクチャの性能を評価するために，理想下限 (Peak Latency) と演算器・パイプラインのレイテンシを含む実行サイクル数 (Real Latency) を導出し，実効性能/理論性能（sustained-to-peak) を評価指数の一つとしている．この Peak Latency と Real Latency は次章：ベースライン実装 (先行研究の再現) の議論に必要なため，ここで説

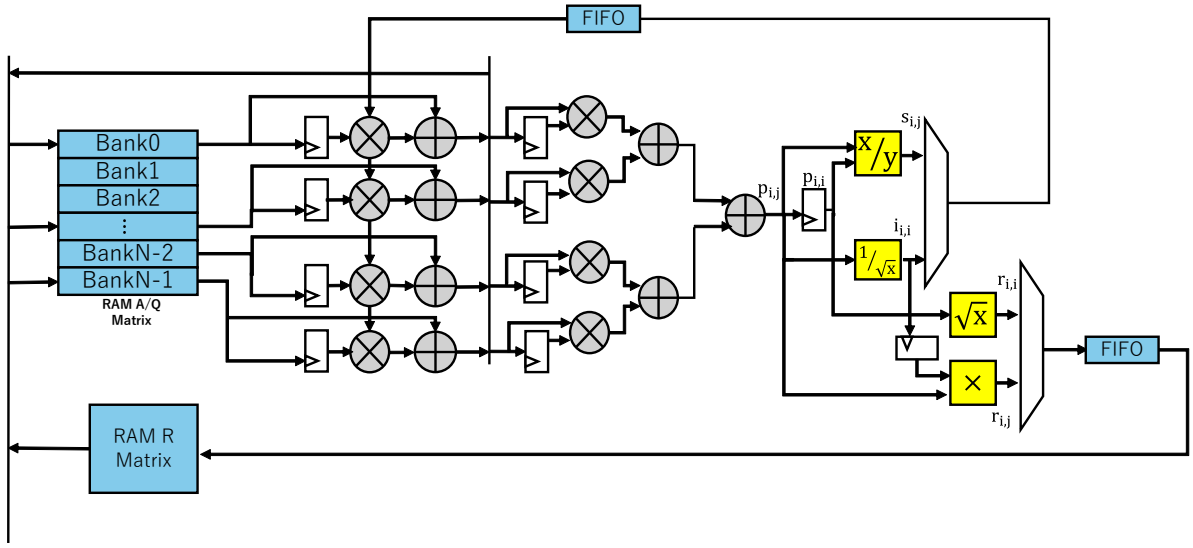


図 3.1　先行研究のアーキテクチャ図（[1] の Figure 1 を基に作成）

明する．

まず，Peak Latency の導出方法について説明する．先行研究のアーキテクチャでは 1 サイクルに 1 つ列ベクトルを処理する．すると，QRD 自体の演算量は $\boldsymbol{O(n^3)}$ でも，列を読む回数に支配されるので，実装上のステップ数は $\boldsymbol{O(n^2)}$ になる．先行研究の Algorithm 3 より，外側ループの反復回数は n 回．また，外側ループのイテレーションが進むと読み出す列ベクトルは 1 ずつ減る．したがって，パイプライン遅延を無視すると理想的な最小サイクルは次式 3.2 で表すことができる．

$$PeakLatency = \sum_{k=1}^{n} k = \frac{n(n+1)}{2} \tag{3.2}$$

次に Real Latency の導出方法について説明する．先行研究のアーキテクチャでは内積計算の計算結果が，除算や逆平方根の演算を通過する．そして，FIFO を経由して乗算-減算器の入力になる．したがって，次反復を開始するためのレイテンシは内積計算器，除算器，配線のためのパイプライン遅延に左右される．先行研究ではそのレイテンシを vectorDepth として次式 3.3 で表している．

$$vectorDepth = 8 + (4 + 3\lceil \log_2(n) \rceil) + 15 + x \tag{3.3}$$

ここで，n は行列の次数をあらわす．15 は除算器のレイテンシ，$(4 + 3\lceil \log_2(n) \rceil)$ の項は内積計算のレイテンシ，8 は乗算-減算器の往復レイテンシ，x は追加のフィッティング用パ

イプラインとして vectorDepth を計算している．この vectorDepth を定義したうえで，Real Latency は次式 3.4 で表される．

$$t_{QRD} = \sum_{i=1}^{n} max(i, vectorDepth) \tag{3.4}$$

i は各反復で流れる列ベクトルの数を表す．先行研究の実装では読み出す列ベクトルの数が i が進むごとに 1 減る．したがって，列の数が vectorDepth より小さい場合，vectorDepth のサイクル数分待つ必要がある．逆に，列の数が vectorDepth より大きい場合は，vectorDepth のサイクル数を隠すことができる．よって，式 3.4 で Real Latency を表すことができる．以上により，Peak Latency と Real Latency を求めることができる．

先行研究の開発環境

ここに先行研究の開発環境を記す．

- FPGA : Altera Arria 10 (-l1)
- 開発ツール : Intel Builder Advanced
- 言語: C++

3.1.4 先行研究の性能・リソース

以下に先行研究のリソースの表 3.2，性能の表 3.1， 各演算器ごとのレイテンシの表 3.3 を載せる．また，演算器ごとのレイテンシ表 3.3 は論理合成時に得ることができる．このレイテンシから，式 3.2 と 3.4 を用いて vectorDepth と Real Latency を導出する．そして，3.1 の Performance は， 周波数と Real Latency から式 3.5 を用いて導出したものである．

先行研究の性能では，表 3.1 に記す通り，$N \geq 256$ において sustained-to-peak が 95% をこえる結果になっている．

したがって，先行研究の手法では Algorithm 2 で課題としていたデータ依存とノルム計算のレイテンシを改善し，sustained-to-peak が 95% を超える性能を達成した．

$$t_{\text{core}}[\mu s] = \frac{\text{Real Latency [cycle]}}{f[\text{MHz}]} \tag{3.5}$$

表 3.1 Sustained-to-Peak and absolute performance より抜粋 [1]

n	Latency [cycle]		sustained-to -peak	Performance	
	Peak	Real		GFLOPs	μs
64	2080	3355	61%	71.7	7.27
128	8256	9741	84%	190.9	21.8
256	32896	34607	95%	417.8	80.4
385	73920	75690	97%	577.6	195
512	131328	133408	98%	744.2	359.5

表 3.2 先行研究のリソース表（Table 2 より抜粋）[1]

行列サイズ	Results			
	周波数 [MHz]	ALM	DSP	M20K
64	461	4,808	135	72
128	445	11,120	265	136
256	430	22,013	526	264
384	388	32,726	787	392
512	371	43,404	1,047	520

表 3.3 先行研究における各演算器レイテンシの比較（Table 1 より抜粋）[1]

行列サイズ	レイテンシ [cycle]（先行研究）			
	divide	rsqrt	multiply-substract	dot
64	17	11	4	26
128	17	11	4	30
256	17	11	4	34
512	17	11	4	38

第 4 章

ベースラインの実装

本章では本章では，提案手法の比較基準として，先行研究 [1] のアーキテクチャを可能な範囲で再現実装する．以降，本章の実装を Baseline と呼ぶ.

4.1 再現方針と対象

4.1.1 再現の目的

本研究では，提案手法の有効性を定量的に示すため，比較基準（ベースライン）として先行研究 [1] が提案する MGS-QRD 向け FPGA アーキテクチャを可能な範囲で再現実装する．ここで再現の対象は，以下から構成される FPGA 側アーキテクチャ全体である．

1. 演算パイプライン（内積・乗算減算・平方根/除算等）
2. オンチップメモリ構成とデータ配置
3. モジュール間のデータフロー（FIFO/遅延調整を含む）
4. それらを統合したカーネル全体のスケジューリング

なお，先行研究ではホスト–FPGA 間 I/O の詳細が十分に明示されないため，本研究ではホスト I/O を除く FPGA 側のアーキテクチャを再現範囲とし，測定も同範囲に限定する．

4.1.2 再現条件（数値形式・行列サイズ）

数値形式は先行研究と同様に単精度浮動小数点（FP32）を前提とする．評価対象の行列次数は $N \in \{64, 128, 256, 512\}$ とする．

4.1.3 再現範囲と差異

先行研究ではホストと FPGA 間のデータ転送タイミングやインタフェース実装の詳細が明示されないため，本研究ではホスト–FPGA 間 I/O を含むシステム統合部分は再現範囲外とし，FPGA カーネル内部（演算パイプラインおよびオンチップメモリ構成）を再現範囲とする．また，ターゲットデバイスおよび設計ツールフローが先行研究と異なるため，完全な一致（配置配線や Fmax の同一化）は保証できない．このため，再現実装は「演算構造とデータフローの同等性」を優先し，差異は以降の節で明示する．

4.1.4 再現度の評価指標

再現度は次の指標により評価する．

1. 使用リソース (DSP,ALM)
2. 動作周波数 (Fmax)
3. 実行サイクル数 (Real Latency)

これらの指標を用いて，先行研究の報告値に対してどの程度近い実装となったかを示す．なお, 先行研究のリソースと本研究のリソースは, 外部メモリ I/F やインターコネクトを含む範囲が一致しない可能性があるため,DSP 数以外のリソース比較は参考値として扱い, 主に Fmax および実行サイクル,DSP 数に基づいて評価する．

4.2 実装

4.2.1 設計・開発環境

本研究では，Intel oneAPI を用いて DPC++（SYCL）でカーネルを記述し， icpx コンパイラにより RTL（HDL）を生成した後，Quartus Prime により論理合成・配置配線を行い，FPGA ボード上で動作検証を行う．以下に設計・開発環境をまとめた表 4.1 を記す．

4.2.2 アーキテクチャの対応付け

本節では，先行研究のアーキテクチャと，本研究で構築したベースライン実装のアーキテクチャの対応関係を述べる．先行研究は，MGS-QRD の各計算段を複数の演算ブロックに分割し，ブロック間をストリーミングに接続することで高スループット化している．

表 4.1 設計・開発環境

項目	内容
FPGA ボード	IPAC-1000（Agilex™ 7 M-Series AGMF039R47A2E2V 搭載）
ホスト接続	PCIe Gen5 x16
設計ツール	Intel oneAPI Base Toolkit 2025.0
論理合成	Quartus Prime Pro 25.1.0
記述言語	DPC++（SYCL ベース）

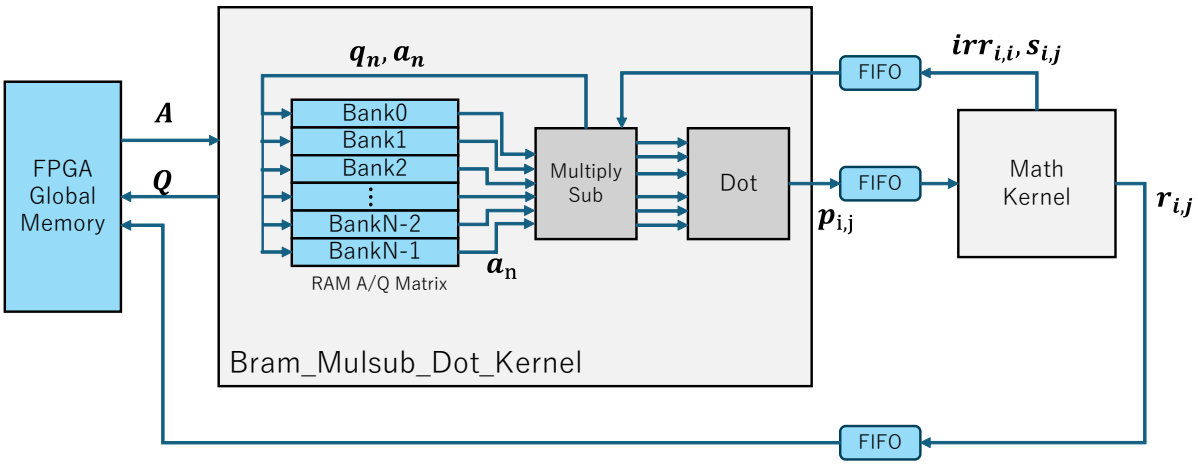


図 4.1 ベースラインのカーネル構成

本研究では，このデータフロー構造を Intel oneAPI 上で再現するため，カーネル間接続に `pipe`（FIFO ストリーム）を用いる．

ベースライン実装は図 4.1 に示す,2 つの主要カーネルから構成される．第 1 のカーネル (Bram_Mulsub_dot_Kernel) は，行列 A および直交化途中の Q を保持するオンチップ RAM と，乗算減算器および内積計算器を含み，MGS における直交・更新計算（例：$a_j \leftarrow a_j - s_{i,j}a_i$）と内積生成を担当する．第 2 のカーネル (Math_Kernel) は，除算，平方根，逆平方根および補助的な乗算器から構成され，正規化係数の算出やスケーリング等の初等関数演算を担当する．

両カーネルはそれぞれ内部をパイプライン化し，さらに `pipe` により結果を逐次受け渡すことで，カーネル間も含めてストリーミングパイプラインとして動作するように設計する．これにより，先行研究の「演算ブロック分割＋ストリーミング接続」という全体アーキテクチャを oneAPI 上で対応付ける．

oneAPI における実装方針と最適化指示子

ベースライン実装では, 先行研究の資源構成およびスループットに近づけるため, oneAPI が提供する各種指示子（属性）を用いて，演算器割当とメモリ構成を制御する．以下に主要な指示子を示す.

- math_dsp_control：演算の実装先（DSP 優先/Softlogic 優先）を指示する
- fpga_memory: 配列のブロック RAM か MLAB を指定する
- max_replicates：コンパイラによるメモリバンクの複製を防ぐ意図で付与する（完全にゼロ複製を保証はされない）
- simple_dual_port：読み/書きポート構成を明示する
- bank_bits/bankwidth：メモリバンク分割を誘導する
- fpga_register：レジスタ化を誘導する
- ivdep: ループ依存の誤検出を避けて II の低減を狙う

用語補足（II）: II（Initiation Interval）は，ループをパイプライン化する際に，新しい反復（iteration）を開始できる最小サイクル間隔を表す．故に, II が小さいほどスループットが高い.

4.3 妥当性の確認

本節では，先行研究のアーキテクチャを再現したベースライン実装が，資源構成および演算器構成の観点で妥当であるかを確認する．具体的には，論理合成後の資源使用量および動作周波数（表 4.2），ならびに主要演算器のレイテンシ（表 6.2）を，先行研究で報告されている値と比較する.

4.3.1 リソース使用量・周波数の比較

表 4.2 は，先行研究に掲載された資源使用量と，本研究ベースライン実装の論理合成結果を示す．前述のとおり，本研究のベースライン実装における資源使用量は，計算コア（演算器・オンチップメモリ）に加え，外部 DDR とのロード／ストアユニット（LSU）およびグローバルメモリとのインターコネクト等のインフラロジックを含む．このため，ALM および M20K は先行研究の報告値よりも大きい結果となっている.

一方，DSP 使用量は行列サイズ $N = 64, 128, 256$ において先行研究と同等であり，

$N = 512$ においても差分は 3 ブロック（Baseline: 1,044，先行研究: 1,047）に留まる．DSP がほぼ一致していることは，乗算減算器および内積計算器といった演算コアの並列度が先行研究と同程度であることを示唆する．また，動作周波数はいずれの行列サイズにおいてもベースラインが先行研究を上回っており，本研究の実装環境（ターゲットデバイスおよびツールフロー）の差異により，高い Fmax が得られたと考えられる．

演算器レイテンシの比較

表 6.2 は，ベースライン実装と先行研究における主要演算器のレイテンシを比較した結果である．multiply-subtract 演算器のレイテンシは全ての行列サイズで 4 サイクルと完全に一致しており，当該演算器のパイプライン段数は先行研究と同等に再現できている．

dot 演算器のレイテンシ差は 3〜6 サイクルの範囲に収まっている一方，除算（divide）はベースラインが 23〜24 サイクルで先行研究（17 サイクル）より 6〜7 サイクル大きい．さらに逆平方根（rsqrt）はベースラインが 23〜24 サイクルであり，先行研究（11 サイクル）と比較して 12〜13 サイクル大きい．この差異は，oneAPI が生成する初等関数演算器（除算・平方根・逆平方根）の実装方式やパイプライン段数が，先行研究の実装（IP 構成）と一致しないことに起因すると考えられる．

以上より，本研究のベースライン実装は，DSP 規模および主要演算器（multiply-subtract, dot）のレイテンシの観点で先行研究と概ね整合しており，先行研究アーキテクチャの再現として妥当な比較基準（Baseline）になっていると判断できる．一方で，ALM/M20K はインフラロジックを含むため増加し，除算・逆平方根のレイテンシには差が残るため，以降の比較ではこれらの差異を考慮して評価を行う．

表 4.2　Baseline 実装と先行研究のリソース比較

行列サイズ	周波数 [MHz]		ALM		DSP		M20K	
	Baseline	先行研究	Baseline	先行研究	Baseline	先行研究	Baseline	先行研究
64	526	461	30,455	4,808	137	135	300	72
128	472	445	40,127	11,120	267	265	366	136
256	451	430	58,945	22,013	526	526	494	264
512	437	371	99,311	43,404	1,044	1,047	754	520

表 4.3　Baseline 実装と先行研究における各演算器レイテンシの比較

行列サイズ	divide [cycle]		rsqrt [cycle]		multiply-substract [cycle]		dot [cycle]	
	Baseline	先行研究	Baseline	先行研究	Baseline	先行研究	Baseline	先行研究
64	24	17	24	11	4	4	29	26
128	24	17	24	11	4	4	34	30
256	24	17	24	11	4	4	37	34
512	24	17	24	11	4	4	44	38

4.4　ベースライン性能

4.4.1　ベースライン性能の比較結果

ベースライン実装の性能を評価するため，先行研究で定義された vectorDepth(式 3.3) および Real Latency(3.4) を用いて，ベースラインにおける $D_v(N)$ と Real Latency を導出した．まず vectorDepth $D_v(N)$ の導出では，先行研究の定義式をそのまま用い，ベースライン実装で得られた各演算器レイテンシ（除算，逆平方根，内積，乗算減算器）を代入して算出した．次に，得られた $D_v(N)$ を用いて，式 3.4 により Real Latency を計算した．

各行列サイズに対して算出した $D_v(N)$ および Real Latency をまとめた結果を表 4.4 に示す．表 4.4 より，ベースライン実装では除算および逆平方根のレイテンシが先行研究よりも大きく，おおむね 23–24 cycle となっている．また，内積計算（dot）のレイテンシも先行研究に比べて増加している．これらの要因によりベースラインの Vector Depth $D_v(N)$ は先行研究より大きくなり，その結果として Real Latency も先行研究に比べてやや増加する傾向が確認できる．

特に小さい行列サイズ（$N = 64, 128$）では，反復数 k が $D_v(N)$ と同程度となる区間が相対的に長く，パイプライン遅延が露出しやすいため，Real Latency の差が効率（Sustained/Peak）に反映されやすい．一方で $N \geq 256$ では，多くの区間で $k \gg D_v(N)$ が成立し，パイプライン遅延が相対的に隠蔽される領域が支配的となる．このため，Real Latency の差は相対的に小さくなり，効率は 93–98% 台に収束する．以上より，ベースライン実装は先行研究に対して初等関数演算器および内積系のレイテンシ差に起因する性能差が生じるものの，行列サイズが大きい領域ではパイプライン化の効果により高い効率を維持できることが示された．

表 4.4　vector Depth および Real Latency の推定値（Baseline と先行研究の比較）

行列サイズ	vector Depth D_v [cycle]		Real Latency T_{real} [cycle]		Sustained/Peak [%]	
	Baseline	先行研究	Baseline	先行研究	Baseline	先行研究
64	61	51	3,910	3,355	52.4	61.0
128	66	55	10,401	9,741	78.8	84.1
256	68	59	35,174	34,607	93.2	94.7
512	76	63	134,178	133,281	97.7	98.3

4.4.2　小括

本章では，先行研究のアーキテクチャを可能な範囲で再現したベースライン実装を構築し，リソース構成および性能指標の観点から整合性を確認した．リソース比較では，DSP 使用量が概ね先行研究と一致しており，内積計算器および乗算–減算器の並列度が同程度となるよう再現できていることを示唆する．また，各演算器のレイテンシ（divide, rsqrt, dot, multiply-subtract）を用いて Vector Depth $D_v(N)$ を導出し，これに基づいて Real Latency および Sustained/Peak を算出した結果，完全に一致するには至らないものの，先行研究と近い Real Latency および高い Sustained/Peak（特に $N \geq 256$ で 93–98% 台）を得た．以上より，本ベースライン実装は以降の提案手法の評価における比較基準として利用可能である．

さらに，動作周波数については全ての行列サイズでベースラインが先行研究を上回った．この要因として，使用ツールチェーンの差異（oneAPI および Quartus の最適化）に加え，ターゲット FPGA が先行研究より新しい世代（Agilex™ 7）であることに起因するアーキテクチャ的な性能差が影響している可能性がある．

第 5 章

提案手法

5.1 先行研究のボトルネック分析

5.1.1 内積計算レイテンシの支配性

修正グラム–シュミット（MGS）に基づく QR 分解では，i 列目の処理において正規化に用いるノルム計算（$r_{i,i}$）および直交化のための係数計算（$r_{i,j} = \langle q_i, a_j \rangle$）が必要となる．これらは本質的に内積（dot-product）を中心とする演算であり，既存研究のアーキテクチャでは，内積演算器のパイプライン深さに起因して，内積計算のレイテンシが全体パイプラインのクリティカルとなりやすいことが報告されている（例：[1]）．本研究で用いるベースライン実装においても，同様に内積計算が支配的となる傾向を確認した．

次に，内積計算の回数を評価する．$n \times n$ 行列に対して，i 列目の処理では $r_{i,i}$（ノルム）に加えて $j = i+1, \ldots, n$ に対する係数 $r_{i,j}$ を計算するため，i ごとの内積回数は $(n - i + 1)$ 回となる．したがって，QR 分解全体で必要な内積回数 N_{dot} は

$$N_{\mathrm{dot}} = \sum_{i=1}^{n} (n - i + 1) = \frac{n(n+1)}{2} \tag{5.1}$$

で与えられる．式 (5.1) より，内積回数は $O(n^2)$ で増加する．さらに，各内積は長さ n のベクトル積和を伴うため，n の増大に伴い内積演算器のパイプライン深さ・処理時間が相対的に無視できなくなる．以上より, 次数が大きいほど内積計算が増加し，内積計算レイテンシはベースライン設計全体レイテンシの支配的要因の一つになり得る．

5.2　本研究で解決する先行研究の課題

前節で述べたように，MGS-QRD における内積計算回数は式 (5.1) で与えられ，行列次数 n の増加に伴い $O(n^2)$ で増大する．さらに，各内積は長さ n の積和演算を含むため，次数が大きい場合には内積計算に要するレイテンシが無視できない．したがって，本研究では，行列次数の増加に伴う内積計算回数の増大，およびそれに起因する全体レイテンシの増加を主要な課題として扱う．

この課題に対し，本提案手法は，1 サイクル当たりに実行可能な内積計算数を増加させることにより，内積計算部分のスループットを向上させ，結果としてベースライン実装の低レイテンシ化を狙う．

5.3　アルゴリズム選択の理由

5.3.1　MGS-QRD の計算・アクセス特性

MGS（Modified Gram–Schmidt）に基づく QR 分解では，行列 $A = [a_1, \ldots, a_n]$ を列ベクトル a_i 単位で逐次直交化する．具体的には，各ステップ i において a_i から正規直交ベクトル q_i を生成し，$j > i$ に対して $r_{i,j} = q_i^{\mathsf{T}} a_j$ を計算した後，$a_j \leftarrow a_j - r_{i,j} q_i$ により残りの列ベクトルを更新する．このように MGS は，処理の基本単位が列ベクトルであり，列ごとの読み出し・更新が繰り返されるため，データアクセスは列指向になりやすい．したがって，行列を列単位（あるいは列を複数バンクに分割）で独立したメモリバンクに格納し，複数要素を並列に読み出す FPGA アーキテクチャへ写像しやすい．

一方，Householder 変換に基づく QR 分解では, 行列に対する行方向・列方向の要素参照が混在している．その結果，メモリアクセスパターンが複雑化し，FPGA 上で高いスループットを維持するためのメモリ構成およびデータ供給機構の設計難度が高くなる．これは先行研究 [1] においても言及されている．

以上より，本研究では列指向アクセスによりメモリ並列化とストリーミング化を行いやすい MGS を採用し，QR 分解の高速化を図る．

5.4 提案設計

5.4.1 内積計算器の二重化

本研究の提案手法は，先行研究 [1] の MGS-QRD 実装において支配的となる内積計算に着目し，j ループにおける内積計算スループットを向上させることを目的とする．Algorithm 3 では，外側ループで i が固定されたとき，内側 j ループにおける各内積 $p_{i,i} = \langle a_{i+1}, a_{i+1} \rangle$ や $p_{i,j} = \langle a_{i+1}, a_j \rangle$ は互いにデータ依存を持たず，独立に計算できる．したがって，j ループの内積計算は並列化可能であり，演算器の複製によってスループット向上が期待できる．具体的には，先行研究で用いられる内積計算器を二重化し，1 回の j イテレーションで 2 つの内積を同時に計算できるようにする．これにより，本提案では j を 2 ずつ更新して処理を進める（$j \leftarrow j + 2$）ことで，j ループの反復回数を従来のおよそ半分に削減し，内積計算に要する総サイクル数の低減を狙う．

5.4.2 オンチップメモリ構成

内積計算器の二重化を有効に機能させるためには，2 つの列ベクトルに対するデータ供給を同一サイクルで行う必要がある．そこで本研究では，ベースラインのメモリ構成を変更し，元行列を偶数列と奇数列に分割してオンチップメモリへ格納する方式 (図 5.1) を採用する．具体的には，$n \times n$ 行列 A の偶数列のみを格納するメモリブロック A_{even} と，奇数列のみを格納するメモリブロック A_{odd} を用意し，オンチップメモリ領域はこれら 2 ブロックから構成される．結果として，オンチップ上には $n \times (n/2)$ の部分行列が 2 つ格納される形となり，偶数列と奇数列に属する列ベクトルを同時に読み出し可能となる．

なお，メモリのバンキング方式は先行研究と同様に行方向（行ごと）に分割する．これにより，列ベクトルを構成する複数要素の並列読み出しを維持しつつ，偶数列と奇数列の 2 ストリームを同時に供給できる．また，先行研究で確立されているカーネル分割および基本的なデータフロー（列ベクトルのロード，内積計算，直交化更新，書き戻し）の枠組みは維持し，内積計算器の二重化とそれを支えるメモリ構成の変更によって，j ループの処理効率を高める．

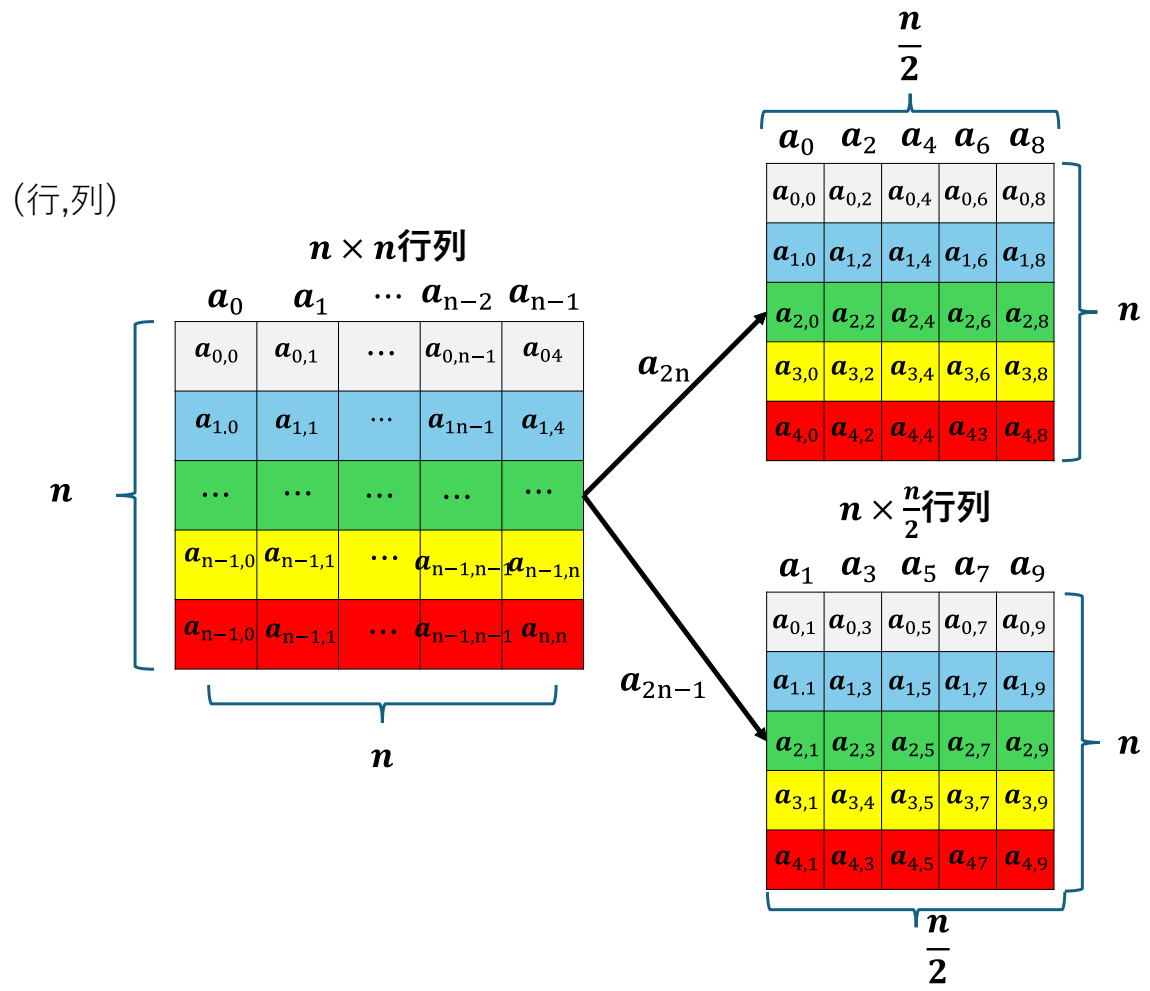


図 5.1　行列の分割方式とバンキング

5.5　提案アーキテクチャ

図 5.2 に提案アーキテクチャを示す．本設計は，先行研究のデータフロー（列ベクトルのバッファリングと，内積計算結果に基づく逐次直交化）を維持したまま，直交化対象の列ベクトルを偶数列・奇数列に分割して並列化する点に特徴がある．提案アーキテクチャは，(i) 偶数列/奇数列を保持するオンチップ RAM，(ii) 乗算–減算（MulSub）器，(iii) 内積（Dot）計算器，(iv) 結果順序を整列するマルチプレクサと FIFO，および (v) 直交化に必要なスカラー量を算出する Math Kernel から構成される．

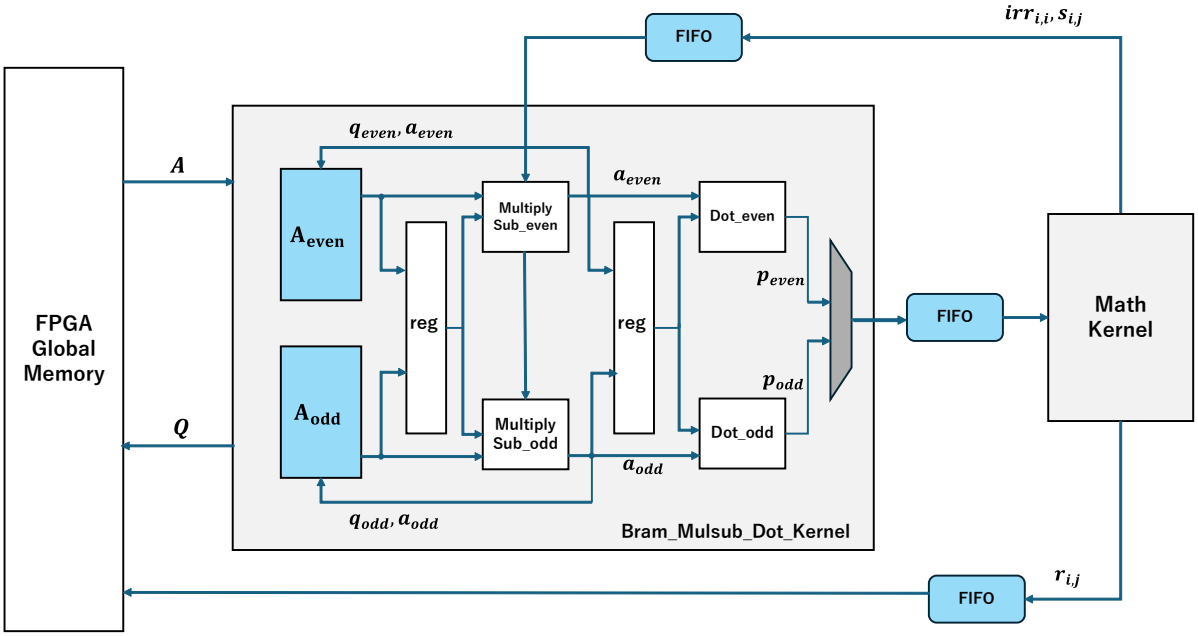


図 5.2　提案アーキテクチャ

5.5.1　FPGA カーネル内部の動作

Bram_Mulsub_Dot_Kernel 内では，入力行列 A の列ベクトルを偶数列用 RAM A_{even} と奇数列用 RAM A_{odd} に分割して保持する．A_{even} は偶数列の列ベクトル，A_{odd} は奇数列の列ベクトルをそれぞれ格納する RAM ブロックである．A_{even} から読み出した列ベクトル（図中 a_{even}）は even 側の演算経路（MulSub_even および Dot_even）に入力され，A_{odd} から読み出した列ベクトル（図中 a_{odd}）は odd 側の演算経路（MulSub_odd および Dot_odd）に入力される．したがって，even 側は偶数列，odd 側は奇数列の列ベクトルのみを担当し，それぞれ独立に計算を進める．

提案設計では，内積計算器（Dot）を 2 基備え，偶数列経路 (even) と奇数列経路 (odd) で独立に内積計算を行う．さらに乗算–減算器（MulSub）も 2 基配置するが，これは単に演算器を増設するためではなく，先行研究で用いられている「列ベクトルをレジスタに保持し，そのレジスタ出力を MulSub へ供給して列更新を行う」データフローを保つためである．すなわち，外側ループの直交化計算において列ベクトルをレジスタ（図中 reg）に格納し，このレジスタ出力を 2 つの MulSub で共有する構成とすることで，偶数列/奇数列それぞれの列更新を同一の流れで実装できる．同様に，内側 j ループにおける内積計算の入力

となる列ベクトル（例えば a_j）もレジスタを介して 2 つの Dot で共有し，各経路は偶数列または奇数列のいずれか一方のみを担当する．

MulSub の出力である更新後の列ベクトルは，偶数列は A_{even} へ，奇数列は A_{odd} へ書き戻される．また，Dot の出力（偶数列側の結果と奇数列側の結果）は，Math Kernel が要求する列順（j の昇順）と整合するよう，マルチプレクサによって送出順序を整列した上で FIFO に書き込み，Math Kernel へ渡す．

5.5.2 FPGA グローバルメモリとのデータ転送

まず初期化段階として，FPGA グローバルメモリ上の行列 A をロードし，各列を偶数列/奇数列に振り分けて A_{even} および A_{odd} へ格納する．直交化の過程で生成される R の要素（例えば $r_{i,j}$）は，後続の同一反復内で参照しない設計方針とし，Math Kernel で算出され次第 FIFO を介してグローバルメモリへ逐次書き戻す．全ての直交化処理が完了した時点で，得られた直交行列 Q をグローバルメモリへ転送する．

5.5.3 アルゴリズムに基づく詳細動作

本節では，図 5.2 の提案アーキテクチャにおける各モジュールの動作を，先行研究 [1] のアルゴリズム（Algorithm 3）の処理順序を参照しながら説明する．提案設計では列ベクトルを偶数列経路（even パス）と奇数列経路（odd パス）に分割し，内側 j ループを $j \leftarrow j+2$ として進めることで，1 回の j イテレーションで 2 つの内積値を生成する．以下では，プレループ（Algorithm 3 の L2–L7）とメインループ（L8 以降）に分けて述べる．

■プレループ時の動作（Algorithm 3: L2–L7） プレループでは，A_{even} および A_{odd} から列ベクトルを順次読み出し，Dot_even と Dot_odd により内積値を計算して FIFO へ送出する．このとき，even パスは偶数列，odd パスは奇数列を担当するため，同一 j イテレーションにおいて even パスが扱う列インデックスは常に odd パスより小さくなる．したがって，FIFO には列順（j の昇順）と整合するよう，even 側で得られた内積値（p_{even}）を先に書き込み，続いて odd 側の内積値（p_{odd}）を書き込む．これにより，Math Kernel は FIFO から読み出す順序を変更することなく，Algorithm 3 に従って $p_{1,1} = \langle a_1, a_1 \rangle$ および $p_{1,j} = \langle a_1, a_j \rangle$（$j \geq 2$）を順次処理できる．

■メインループ時の動作（Algorithm 3: L8 以降） メインループでは, Bram_Mulsub_Dot_Kernel は基本動作として，1 回の j イテレーションごとに even/odd の 2 経路に対して列ベクトル

を 2 本読み出し，Dot 計算結果を 2 個生成して FIFO へ書き込む．同時に，Math Kernel は FIFO を基本的に 2 回読み出し，even/odd 経路へ必要なスカラー値を供給する．

外側ループにおける直交化（Algorithm 3 の $q_i = a_i \times ir_{i,i}$ および $a_j = a_j - s_{i,j}q_i$）では，処理対象となる列 a_i が偶数列か奇数列かにより，更新を担う経路が決まる．具体的には，外側ループのインデックス i が偶数のときは MulSub_even が a_i の正規化および更新を担当し，i が奇数のときは MulSub_odd が担当する．このため，Math Kernel から供給される $ir_{i,i}$ は，i の偶奇に応じて MulSub_even または MulSub_odd へ入力されるように制御する．同様に，$s_{i,j}$（あるいは $r_{i,j}$）などの係数も，対応する列 a_j を処理する経路（even/odd）へ供給される．

■端数列（最後の j イテレーション）に対する処理　提案設計では j を 2 ずつ増加させて処理を進めるため，n が偶数であっても，外側ループの i の偶奇によっては最後の j イテレーションにおいて一方の経路が参照すべき列が存在しない場合がある．例えば，0 始まりのインデックスで偶数列経路が j，奇数列経路が $j+1$ を担当する形で処理しているとき，外側ループの条件によっては終端付近で $(j, j+1) = (n-1, n)$ のような組が生じ，a_n が存在しないため不正な読み出しとなる．そこで本設計では，この端数条件が成立する場合には，列ベクトルおよび FIFO の読み出しを 2 本同時から 1 本へ切り替え，存在する側の列ベクトル（例：a_{n-1}）のみを該当経路へ入力して処理を継続する．これにより，存在しない列ベクトルへのアクセスを回避しつつ，$j \leftarrow j+2$ の並列処理を維持する．

第 6 章

評価

本章では，Baseline 実装（再現実装）と提案手法を比較し，提案手法による性能・コストの変化を定量的に評価する．評価は，設計変更の影響のみが結果に反映されるよう，開発・実装環境および実験条件を Baseline 実装と統一した上で実施した．具体的には，行列サイズ $N = 64, 128, 256, 512$ を対象とし，（i）合成結果として周波数および資源使用量（ALM, DSP，M20K），（ii）周波数と Real Latency から導出した計算コア実行時間，さらに（iii）FPGA グローバルメモリとカーネル間のメモリアクセスと計算を含むカーネル実行時間を比較する．最後に，(iv)CPU や GPU との性能の比較をする． これらの結果を通じて，提案手法の性能について議論する．

6.1 合成結果の比較と考察

図 6.1 は，Baseline 実装と提案手法の合成結果（周波数，ALM，DSP，M20K）を示す．提案手法は演算器並列度を増やす設計変更を含むため，資源使用量の増加と周波数低下が生じる．以降では，DSP と ALM を中心に増加傾向を整理する．

6.1.1 DSP 使用量

DSP 数に着目すると，提案手法は Baseline に対して約 2 倍となった．比率としては 1.94 倍〜1.99 倍 の範囲であり，ほぼ 2 倍に近い増加である．これは，内積計算器および multiply-subtract 系演算器の並列数を増やしたことにより，DSP を主に消費するデータパスが複製されたためと解釈できる．

表 6.1　合成結果の比較（ベースライン vs 提案手法）

行列サイズ	周波数 [MHz]		ALM		DSP		M20K	
	Baseline	提案手法	Baseline	提案手法	Baseline	提案手法	Baseline	提案手法
64	526	478	30,455	35,599	137	266	300	364
128	472	481	40,127	54,611	267	526	366	494
256	451	430	58,945	102,329	526	1,044	494	750
512	437	315	99,311	181,291	1,044	2,080	754	1,262

6.1.2　ALM

ALM は行列サイズの増加に伴い提案手法で大きく増加し，Baseline に対する倍率は $N = 64, 128, 256, 512$ でそれぞれ 1.17 倍，1.36 倍，1.74 倍，1.83 倍 となる．この増加は，演算器の複製によりデータパス周辺の制御・配線・レジスタが増えることに加え，RAM ブロックやメモリアドレス生成・多重化（mux）などの周辺ロジックが増加したことに起因すると考えられる．

6.1.3　周波数への影響

周波数は N=128 を除き提案手法で低下しており，特に N=512 で 437MHz→315MHz と低下が大きい．これは，演算器・メモリ構造の増加に伴って配置配線が複雑化し，クリティカルパスが悪化したためでは，と推測できる．したがって，提案手法の性能評価では，資源増加だけでなく，周波数低下が実行時間に与える影響も併せて議論する必要がある．

6.2　演算器のレイテンシ

表 6.2 に提案手法とベースラインでの各演算器のレイテンシを示す．$N = 256$ の内積計算は 2 サイクル増加し，除算は 1 レイテンシ減少した．この結果からは $N = 256$ 以外は同じサイクル数の演算器が合成できていると考えられる．

表 6.2　Baseline 実装と提案手法における各演算器レイテンシの比較（cycle）

N	divide [cycle]		rsqrt [cycle]		multiply-subtract [cycle]		dot [cycle]	
	Baseline	提案手法	Baseline	提案手法	Baseline	提案手法	Baseline	提案手法
64	24	24	24	24	4	4	29	29
128	24	24	24	24	4	4	34	34
256	24	23	24	24	4	4	37	39
512	24	24	24	24	4	4	44	44

表 6.3　Baseline 実装と提案手法の周波数・Real Latency・計算コア実行時間の比較（差分付き）

行列サイズ	周波数 [MHz]		Real Latency [cycle]		計算コア実行時間 [μs]		
	Baseline	提案手法	Baseline	提案手法	Baseline	提案手法	Δt [μs]
64	526	478	3,910	3,904	7.43	8.17	-0.74
128	472	481	10,401	8,448	22.04	17.56	+4.48
256	451	430	35,174	21,482	77.99	49.96	+28.03
512	437	315	134,178	71,492	307.04	226.96	+80.08

6.3　計算コア実行時間とカーネル実行時間の比較

6.3.1　計算コア実行時間

表 6.3 に周波数と Real Latency，およびそれらから導出した計算コア実行時間を示す．計算コア実行時間は式 3.4 で導出する．

提案手法は $N \geq 128$ において Real Latency が大きく短縮され，周波数低下を考慮しても計算コア実行時間が短縮した結果を確認できた．$N = 128, 256, 512$ において提案手法は Baseline に対してそれぞれ $4.48\mu s$，$28.03\mu s$，$80.08\mu s$ の計算コア実行時間の短縮を達成できた．一方で，$N = 64$ では $0.74\mu s$ だけ増加し，高速化は確認できなかった．これは，$N \geq 128$ が行列サイズの増加に伴うサイクル数（Real Latency）の増加が顕在化する領域であり，提案手法が狙いとする内積計算部のスループット向上が実際に Real Latency の短縮として表れていることを示している．すなわち，周波数低下の影響を加味しても計算コア実行時間が短縮されたことから，内積計算部の並列化（演算器増設）による低レイテンシ化が設計意図どおり達成されたと考えられる．

表 6.4　カーネル実行時間の比較（ベースライン vs 提案手法）

行列サイズ	カーネル実行時間 [ms]		差分	
	Baseline	提案手法	Δt [ms]	削減率 [%]
64	0.56242	0.50551	0.05691	10.1
128	0.98941	0.67129	0.31812	32.2
256	2.970786	1.947920	1.022866	34.4
512	10.58411	7.99240	2.59171	24.5

6.3.2　カーネル実行時間の比較

本研究におけるカーネル実行時間は，Bram_Mulsub_Dot_Kernel と Math_Kernel の起動から，Bram_Mulsub_Dot_Kernel が終了するまでの区間として定義する．また，本計測では 1 回目の測定をウォームアップとして除外し，合計 100 回の計測から得られたカーネル実行時間の平均値を代表値として表 6.4 に示す．

表 6.4 は，FPGA グローバルメモリ ↔ カーネル間のメモリアクセスと計算を含むカーネル実行時間の比較である．こちらでは $N = 64$ を含めて提案手法が短縮している．なお，Baseline 実装と提案手法は，FPGA グローバルメモリとカーネル間のメモリアクセスについて同様の設計方針を採用している．したがって，表 6.4 に示すカーネル実行時間の差は，主として両者のデータパス（とくに内積計算部）における並列度・スループットの差，およびそれに起因するメモリアクセスと計算の重なり方の違いが反映された結果であると考えられる．

6.3.3　GFLOPS の導出

Performance (GFLOPs) の算出

本研究では，MGS（Modified Gram–Schmidt）に基づく QR 分解の演算量を支配項で近似し，総浮動小数点演算数を $F(N) \approx 2N^3$ とおく．ここで 1FLOP は加算・減算・乗算を 1 回とし，FMA は乗算 + 加算 (減算) として 2 FLOPs として数える．計算コア実行時間を T [s] とすると，性能は

$$\text{Perf [GFLOPs]} = \frac{F(N)}{T} \times 10^{-9} = \frac{2N^3}{T} \times 10^{-9} \tag{6.1}$$

$$\text{Perf [GFLOPs]} = \frac{2N^3}{T_{\mu s} \times 1000} \tag{6.2}$$

となる．なお，本定義では sqrt/div は FLOP に含めず，先行研究と同様に add/mul/sub（FMA=2）を主対象として比較を行う．

上記 GFLOPS を表 6.3 の計算コア実行時間と，式 6.2 によって求めた GFLOPS が表 6.5 である．前節で述べた通り，提案手法は計算コア実行時間の短縮に伴って，$N >= 128$ では GFLOPS の性能向上を達成することができた．

6.4 CPU・GPU との比較

本節では，提案 FPGA 実装の性能を，既報の CPU/iGPU/GPU 実装と比較することで，提案手法の位置づけを明確化する．比較対象として，MGS（Modified Gram–Schmidt）に基づく QR 分解をプロセッサ統合型 GPU（iGPU）で評価した研究，および GPU 上でバッチ QR 分解を評価した 2 件の研究を取り上げる．

まず，[20] は，プロセッサ統合型 GPU（iGPU）を用いた MGS によるベクトル直交化（QR 分解に相当）を対象とし，Intel Haswell（Gen 7.5 iGPU）および Skylake（Gen 9 iGPU）プロセッサ上で OpenCL により MGS アルゴリズムを実装・評価したものである．本研究では，同論文 Fig.1 に示される CPU（Skylake Seq）および iGPU（Gen9 Basic）の実行時間を図から概算し，提案 FPGA 実装のカーネル実行時間と比較する．

次に，[21] は，GPU 上でバッチ QR 分解を実装し，階層行列圧縮への応用を念頭に性能評価を行った研究である．本研究では，同論文に示される QR 分解の性能グラフから GFLOPS（あるいはそれに準ずる性能指標）を概算し，提案 FPGA 実装の GFLOPS と比較する．

さらに，[22] は，NVIDIA Tesla A100 GPU を用いたバッチ QR 分解の設計・最適化を扱

表 6.5 Baseline 実装と提案手法の GFLOPS 比較

行列サイズ	GFLOPS	
	Baseline	提案手法
64	70.6	64.2
128	190.3	238.9
256	430.2	671.6
512	874.3	1182.7

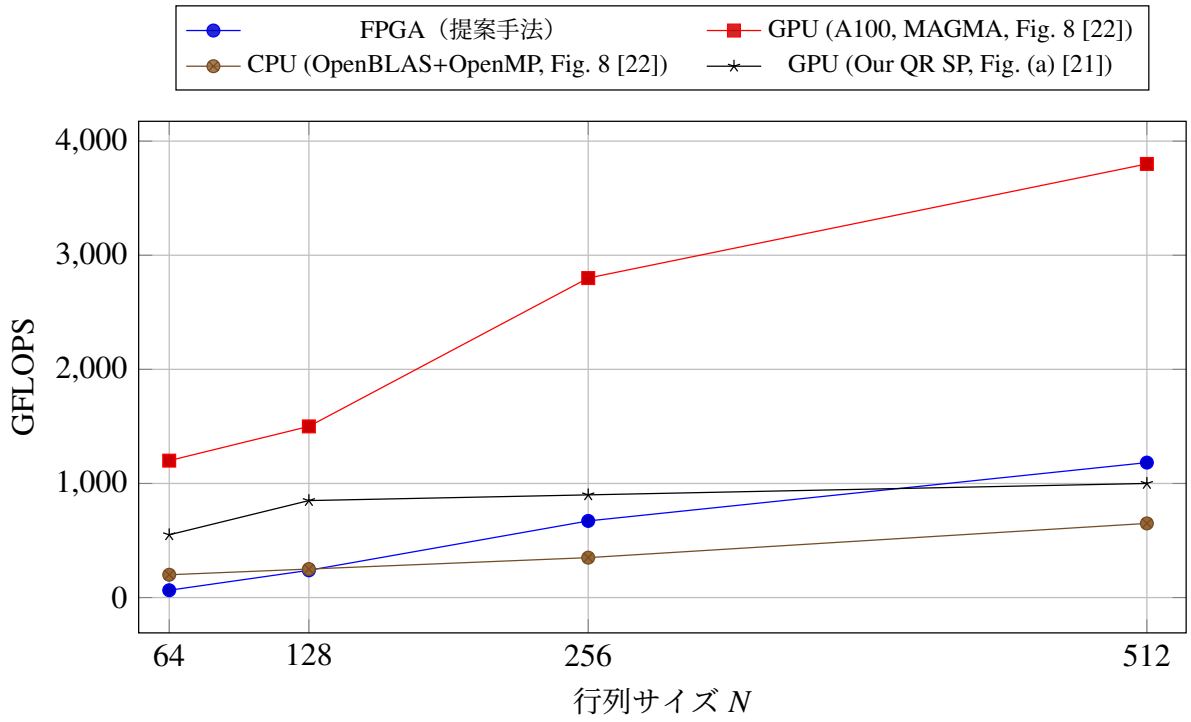


図 6.1　GFLOPS の比較（文献値は各図からの概算）

い，正方行列（square matrices）に対する最終的な性能結果を示している．本研究では，同論文 Fig.8（単精度側）の GFLOPS を図から概算し，提案 FPGA 実装との比較に用いる．

なお，本節で引用した文献値はいずれも単精度浮動小数点（single precision）実装の結果である．ただし，文献値は図からの概算値であり，厳密な再現値ではない点に留意する．

異種ハードウェアとの比較結果を図 6.2 および図 6.1 に示す．実行時間に着目すると，提案手法（FPGA）は $N \geq 128$ において，GPU（A100）を除く比較対象（CPU および iGPU）に対して短い実行時間を達成していることが確認できる．一方で，GPU（A100）に対しては実行時間の優位性を示すには至らず，最も高い性能は GPU（A100）が示した．

GFLOPS についても同様の傾向が見られ，提案手法は GPU（A100）に比べて小さい値となった．ただし，$N = 512$ では GPU（A100）以外の文献値を上回る一方，$N = 256$ では他の GPU 手法（Our QR SP）が提案手法を上回る結果となった．

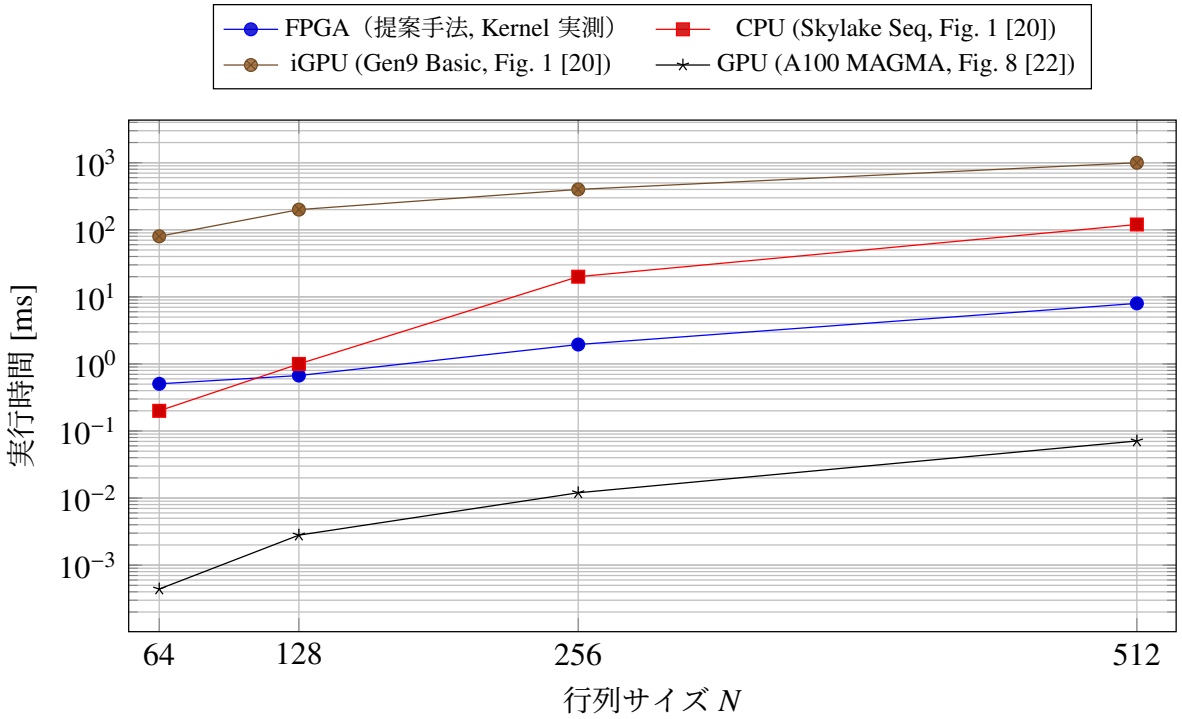


図 6.2 実行時間の比較（FPGA は Kernel 実測，文献値は図から概算．A100 は GFLOPS から $2N^3$ で換算した参考値）

第 7 章

結論

7.1 結論

本研究では，MGS-QR 分解の低レイテンシ化を目的として，内積計算のスループット向上に着目した FPGA 実装を提案した．提案手法では，ベースラインに対して計算コア実行時間およびカーネル実行時間の両方で実行時間を短縮できることを確認した．具体的には，計算コア実行時間においては $N \geq 128$ の条件下で $4\,\mu s$〜$80\,\mu s$ の短縮を達成し，カーネル実行時間においても $0.05\,ms$〜$2.59\,ms$ の削減を達成した．これらの結果より，提案手法は狙い通り，内積計算の処理能力改善に起因する低レイテンシ化を実現したといえる．

一方で，合成結果からは提案手法の設計意図通り，内積計算器および乗算一減算器を各 1 基追加したことにより，DSP 使用量がベースライン比で 2 倍となることを確認した．また，演算器の複製に伴い制御の追加や配線の複雑化が生じ，ALM 数の増加も確認された．したがって，本提案は「レイテンシ短縮」と「資源増加（特に DSP）および制御・配線に起因する ALM 増」のトレードオフの上に成立している．

CPU および GPU との比較では，提案手法が CPU 実装 [20] と GPU 実装 [21] に対して，一部条件下では GFLOPS が優勢である．以上より，提案手法はベースライン FPGA 実装に対しては有効に低レイテンシ化を達成した一方で，GPU と同等以上の性能を得るには，並列度・メモリアクセス・カーネル構成（例：転送と計算の重畳，カーネル統合，メモリ帯域の活用）など，システム全体の最適化が今後の課題である．

7.1.1 今後の課題

本研究では提案手法により MGS-QR 分解の高速化を示したが，さらなる性能向上およびボトルネックの明確化に向けて，以下の課題が残る．

1. **内積計算器の増設によるスケーリング評価と律速要因の同定**
   使用した FPGA ボード（IPAC-1000）には未使用リソースが残存している．したがって，内積計算器（ドット積演算器）を可能な限り増設した場合の到達性能を評価し，性能が頭打ちとなる要因（例：メモリ帯域，配線混雑による達成周波数の低下，パイプラインのストール伝搬等）を系統的に調査する必要がある．これにより，現状の設計が演算律速かメモリ律速かを定量的に切り分けられる．
2. **達成周波数向上オプションを有効化可能なアーキテクチャの検討**
   Intel oneAPI には達成周波数（Fmax）の改善に寄与する最適化オプションが存在するが，本提案手法では当該オプションを有効化できなかった．今後は，当該オプションの適用を阻害している構造的要因（例：カーネル間依存，制御ハザード，ハンドシェイクやバックプレッシャ，ループ構造やメモリアクセス形態）を分析し，オプション適用と両立するアーキテクチャへ再設計することで，周波数改善と実行時間短縮を図る必要がある．
3. **グローバルメモリ帯域の利用効率解析と HBM 活用設計への展開**
   提案手法が FPGA のグローバルメモリ帯域をどの程度活用できているかについて，定量的な解析が不十分である．実効帯域（GB/s）と理論帯域の比を評価し，バースト長，アクセスの連続性，同時アクセス数，競合（バンク競合・ポート競合）などが帯域利用率を制限している要因を明らかにする必要がある．さらに，帯域律速が支配的であると判明した場合には，ipac-1000 に搭載される HBM の利用を前提としたデータ配置（ストライピング）および並列転送・計算重畳の設計を検討し，メモリサブシステム起因の性能限界を押し上げることが重要である．

謝辞

本研究の開発環境・FPGA は IBEX テクノロジー株式会社との共同研究によるものです．

初めに，学部４年生から修士２年生までの３年間，大変お世話になりました，佐藤真平先生に深く感謝の意を伝えます．先生の研究室に所属しなければ，回路設計や FPGA などに興味を抱くことはなかったと思います．私の就職先につきましても，先生とのご縁がなければ絶対になかったものだと思います．自分でもハードウェア関係の会社に就職するとは全然思いませんでした．あと，量子化学計算という壮大なテーマに触れることもなかったと思います．学年が上がるごとに，研究を通して自主性をもって様々なことを学ぶことの大切さに気付く日々でした．本当にありがとうございます．私が就職した後も，IBEX の関係で何かしらお付き合い等があると思いますが，その時はよろしくお願いいたします．

副査である，上口光先生と冨田孝幸先生にも深く感謝の意を伝えます．上口先生には副指導員としてもご指導賜り，輪講や学部での実験，卒業発表に至るまで幅広くお世話になりました．ここに深く感謝申し上げます。冨田先生は，学部での実験においてご指導頂くとともに，学内ですれ違う折にいつも楽しくお話しくださり，大変お世話になりました．ここに深く感謝申し上げます．

IBEX テクノロジー株式会社の望月様，松永様にも深く感謝の意を伝えます．開発環境の準備や IPAC に関する質問などいつも迅速に答えて頂きありがとうございます．去年の 10 月の研修につきましても，私の研究を見てくださりありがとうございました．お二人の対応の早さや優しさで研究を乗り切ることができました．ありがとうございます．

HPC システムズの本田様にも深く感謝の意を申し上げます．量子化学計算という難しい分野の話を分かりやすく，面白く教えて頂きありがとうございました．本田様の講義がなければ，今の理解度に絶対至らなかったと思います．ありがとうございます。

研究室の同期である成田豊隆君にも多大なる感謝の意を伝えます．貴殿とお互いを鼓舞し，励まし，頑張ることでこの修士論文を書くに至ることができました．修士卒業を目指し，共に研究生活での楽しさや苦労を共有できたことは一生の思い出です．貴殿が同期で

本当に良かったです．ありがとうございます。

研究室の修士１年、学部４年の皆様にも深く感謝の意を伝えます．特に修士１年の学生達につきましては，この２年間で研究室以外でも，たくさん遊び，食を共にすることができて本当に楽しかったです．あなた達の存在が研究のモチベーションや研究室に来る理由の一つに間違いなくなっていました．来年もし私が長野に来ることがあれば，その時は是非一緒にご飯にでも行きましょう．

参考文献

[1] Martin Langhammer and Bogdan Pasca. High-performance QR Decomposition for FPGAs, journal = Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA '18), year = 2018, pages = 183–188, doi = 10.1145/3174243.3174273, publisher = Association for Computing Machinery, address = New York, NY, USA.

[2] 秀俊野田. Python で動かして始める量子化学計算. コロナ社.

[3] Vladimir Mironov, Yuri Alexeev, Kristopher Keipert, Michael D'mello, Alexander Moskovsky, and Mark S. Gordon. An efficient mpi/openmp parallelization of the hartree-fock method for the second generation of intel® Xeon Phi™ processor. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '17)*, pp. 1–12, New York, NY, USA, 2017. Association for Computing Machinery.

[4] Pinghui Mo, Yujia Zhang, Zhuoying Zhao, Hanhan Sun, Junhua Li, Dawei Guan, Xi Ding, Xin Zhang, Bo Chen, Mengchao Shi, Duo Zhang, Denghui Lu, Yinan Wang, Jianxing Huang, Fei Liu, Xinyu Li, Mohan Chen, Jun Cheng, Bin Liang, Weinan E, Jiayu Dai, Linfeng Zhang, Han Wang, and Jie Liu. High-speed and low-power molecular dynamics processing unit (mdpu) with ab initio accuracy. *npj Computational Materials*, Vol. 10, No. 1, p. 253, 2024. Article number: 253; pp. 1–10.

[5] Attila Szabo and Neil S. Ostlund. 新しい量子化学 : 電子構造の理論入門. 上. 東京大学出版会, 東京, 1987 年 7 月. 原タイトル: Modern quantum chemistry.

[6] Yukihiro Hasegawa, Jun ichi Iwata, Miwako Tsuji, Daisuke Takahashi, Atsushi Oshiyama, Kazuo Minami, Taisuke Boku, Fumiyoshi Shoji, Atsuya Uno, Motoyoshi Kurokawa, Hikaru Inoue, Ikuo Miyoshi, and Mitsuo Yokokawa. First-principles calculations of electron states of a silicon nanowire with 100,000 atoms on the K computer.

In *Conference on High Performance Computing Networking, Storage and Analysis, SC 2011, Seattle, WA, USA, November 12–18, 2011*, pp. 1:1–1:11. ACM, 2011.

[7] Fusong Ju, Xinran Wei, Lin Huang, Andrew J. Jenkins, Leo Xia, Jia Zhang, Jianwei Zhu, Han Yang, Bin Shao, Peggy Dai, David B. Williams-Young, Ashwin Mayya, Zahra Hooshmand, Alexandra Efimovskaya, Nathan A. Baker, Matthias Troyer, and Hongbin Liu. Acceleration without disruption: DFT software as a service. *Journal of Chemical Theory and Computation*, Vol. 20, No. 24, pp. 10838–10851, 2024.

[8] Inkoo Kim, Daun Jeong, Leah P. Weisburn, Alexandra Alexiu, Troy Van Voorhis, Young Min Rhee, Won-Joon Son, Hyung-Jin Kim, Jinkyu Yim, Sungmin Kim, Yeonchoo Cho, Inkook Jang, Seungmin Lee, and Dae Sin Kim. Very-large-scale GPU-accelerated nuclear gradient of time-dependent density functional theory with tamm–dancoff approximation and range-separated hybrid functionals. *Journal of Chemical Theory and Computation*, Vol. 20, No. 20, pp. 9018–9031, 2024.

[9] Yosuke Oyama, Takumi Honda, Atsushi Ishikawa, and Koichi Shirahata. Accelerating hybrid DFT simulations using performance modeling on supercomputers. In *23rd IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid 2023), Bangalore, India, May 1–4, 2023*, pp. 524–534. IEEE, 2023.

[10] Zhaolong Luo, Xinming Qin, Lingyun Wan, Wei Hu, and Jinlong Yang. Parallel implementation of large-scale linear scaling density functional theory calculations with numerical atomic orbitals in HONPAS. *Frontiers in Chemistry*, Vol. 8, p. 589910, November 2020.

[11] Sambit Das, Phani Motamarri, Vishal Subramanian, David M. Rogers, and Vikram Gavini. Dft-fe 1.0: A massively parallel hybrid cpu-gpu density functional theory code using finite-element discretization. *Computer Physics Communications*, Vol. 280, No. 1, p. 108473, August 2022.

[12] Victor Wen-zhe Yu and Marco Govoni. Gpu acceleration of large-scale full-frequency gw calculations. *Journal of Chemical Theory and Computation*, Vol. 18, No. 8, pp. 4690–4707, August 2022.

[13] 長谷川秀彦, 他. 計算力学レクチャーコース固有値計算と特異値計算. 丸善出版, 2019.

[14] 英晴天野（編）. FPGA の原理と構成. オーム社, 2016 年 4 月. 発売日: 2016-04-22.

[15] Altera. *Agilex™ 7 FPGAs and SoCs Device Overview*, 2025. Version: current, accessed 2026-01-27.

[16] Altera. *Agilex™ 7 Variable Precision DSP Blocks User Guide*, 2025. Online; accessed

2026-01-27.

[17] Intel Corporation. Intel® oneapi programming guide. https://cdrdv2-public.intel.com/848694/oneapi_programming-guide_2025.1-771723-848694.pdf, March 2025. Version 2025.1 (Document ID 771723), published 2025-03-31, accessed 2026-02-01.

[18] Intel Corporation. Intel® oneapi dpc++/c++ compiler handbook for fpgas. https://cdrdv2-public.intel.com/829749/oneapi-fpga-add-on_developer-guide_2025.0-785441-829749.pdf, May 2025. Version 2025.0 (Document ID 785441), published 2025-05-05, accessed 2026-02-01.

[19] Martin Langhammer. QRD for parallel arithmetic structures. *24th IEEE Symposium on Computer Arithmetic (ARITH 2017)*, pp. 146–147, July 2017.

[20] Thomas Jakobs, Lukas Reinhardt, and Gudula Rünger. Performance and energy consumption of a gram–schmidt process for vector orthogonalization on a processor integrated GPU. *Sustainable Computing: Informatics and Systems*, Vol. 29, p. 100456, 2021.

[21] Wajih Halim Boukaram, George Turkiyyah, Hatem Ltaief, and David E. Keyes. Batched QR and SVD algorithms on GPUs with applications in hierarchical matrix compression. *Parallel Computing*, Vol. 74, pp. 19–33, 2018.

[22] Ahmad Abdelfattah, Stan Tomov, and Jack Dongarra. Batch QR factorization on GPUs: Design, optimization, and tuning. In *Computational Science – ICCS 2022*, Vol. 13350 of *Lecture Notes in Computer Science*, pp. 60–74. Springer, 2022.